



國立臺北科技大學

資訊工程系碩士班

碩士學位論文

**一個由 LLM 驅動之 AI 代理人透過操作
API 自動完成任務之研究**

**A Study on LLM-Driven AI Agents for Automated
Task Execution through API Calls**

研究生：吳柏諭

指導教授：劉建宏 博士

中華民國一百一十四年七月



國立臺北科技大學

資訊工程系碩士班

碩士學位論文

**一個由 LLM 驅動之 AI 代理人透過操作
API 自動完成任務之研究**

**A Study on LLM-Driven AI Agents for Automated
Task Execution through API Calls**

研究生：吳柏諭

指導教授：劉建宏 博士

中華民國一百一十四年七月

國立臺北科技大學
研究所碩士學位論文口試委員會審定書

本校資訊工程研究所吳柏諭君

所提論文，經本委員會審定通過，合於碩士資格，特此證明。

學位考試委員會

委

員：

謝文龍

吳和庭

劉建宏

指導教授：

劉建宏

所

長：

劉建宏

中華民國 一百一十四 年 七 月 十七 日

摘要

關鍵詞：大型語言模型、工具調用、代理人

大型語言模型 (Large Language Model, LLM) 具備與人類類似的思考與推理能力，逐漸被應用在代理人核心以執行多步驟應用程式介面 (Application Programming Interface, API) 調用。現有方法大多是將所有 API 文件與過往執行紀錄使用上下文學習 (In-Context Learning) 方法提供給 LLM，由其自行決定是否與如何使用 API。或是透過微調 (fine-tuning) 使 LLM 內建 API 知識，輸出時自動生成並執行 API 呼叫。然而這些方法面臨了諸多挑戰，包含 LLM 輸入上下文的限制，難以同時處理大量 API 文件與過往記憶；使用微調成本高且不易因應 API 變動；以及特殊 API 依賴關係常導致模型忽略必要步驟而產生幻覺等。為解決上述提到的問題，本論文提出一種無需微調的代理人框架，稱為 APIKGagent。此框架使用知識圖譜 (Knowledge Graph, KG) 作為 LLM 的輔助核心，建構 API 節點與其輸入、輸出的依賴關係於知識圖譜內。於任務執行時，APIKGagent 會先根據使用者指令透過知識圖譜搜尋潛在相關 API 以縮小輸入範圍，再進行任務規劃，將任務拆解為多個步驟，並為每步驟選擇合適 API。針對具依賴關係的 API，APIKGagent 會自動依照知識圖譜中定義的順序預先取得所需資源，避免 LLM 幻覺產生錯誤參數。為驗證本方法之有效性，本論文於 Appworld 模擬環境進行測試，Appworld 涵蓋 9 個服務共 457 支 API，可模擬現實使用場景。評估標準為任務是否完成並成功觸發預期狀態變化，結果顯示本方法在無需微調的情況下於 Appworld test_normal 資料集達到 41.7% 任務成功率。儘管仍具挑戰，但實驗證實知識圖譜輔助 API 選取與規劃具可行性，尤其適用於高 API 數量與複雜依賴情境。

Abstract

Keyword: Large Language Model, Tool Calling, Agent

Large Language Models (LLMs), with their human-like reasoning and cognitive capabilities, are increasingly being applied as agents for multi-step Application Programming Interface (API) invocations. Prevailing methods provide the LLM with all API documentation and historical execution records through In-Context Learning (ICL), allowing the model to autonomously decide whether and how to use the APIs. Alternatively, fine-tuning is used to embed API-specific knowledge into the LLM, enabling it to automatically generate and execute API calls. However, these approaches face some challenges, including the context length limitations of LLMs, which hinder the processing of extensive API documentation and memory; the high cost and poor adaptability of fine-tuning to API changes; and hallucination, where complex API dependencies lead to the omission of necessary steps. To address these issues, this thesis proposes a tuning-free agent framework named APIKGagent. This framework leverages a Knowledge Graph (KG) as an auxiliary core for the LLM, structuring API nodes and their input-output dependencies within the graph. During task execution, APIKGagent first queries the KG based on the user's instruction to retrieve a narrowed-down set of relevant APIs. It then proceeds to task planning, decomposing the task into multiple steps and selecting the appropriate API for each. For APIs with inter-dependencies, APIKGagent automatically retrieves the required resources in the sequence defined by the KG, preventing the LLM from hallucinating erroneous parameters. We validated our method in the Appworld simulation environment, which comprises 457 APIs across 9 services, mimicking real-world scenarios. Performance was evaluated based on task completion, defined as the successful triggering of the intended state change. The results show that our approach, without any fine-tuning, achieves a 41.7% task success rate on the Appworld test_normal dataset. Although challenges remain, our experiments confirm the viability of using a Knowledge Graph to assist in API selection and planning, proving particularly applicable to scenarios with a high volume of APIs and complex dependencies.

致謝

本論文得以順利完成，首先由衷感謝指導教授劉建宏老師。在碩士班的求學過程中，承蒙老師的悉心指導與諄諄教誨，不僅在學術研究上給予我莫大的啟發與支持，更在待人處事上使我獲益匪淺。老師嚴謹的治學態度與無私的奉獻精神，將是我未來學習的典範。

此外，亦感謝梁文耀教授。感謝您在每次的會議討論中，耐心聆聽我的報告，提供諸多寶貴的建議，使本論文的架構更完善，方向更加明確。

感謝實驗室的建澄、祐清、俊陞、陳謙、鈺亭、亮節、庭維、喬晟、書廷、鈺婷、派軒等全體同學。感謝你們在研究路上的相互扶持與鼓勵，共同切磋琢磨的時光，不僅讓研究生生活充滿樂趣，也豐富了我的碩士生涯。

最後，謹將本論文獻給我的家人。感謝你們一直以來的支持、包容與關愛，做我最堅實的後盾，在我壓力大時給予我支持與鼓勵，使我能無後顧之憂地專注於學業，順利完成碩士學位。



目錄

摘要	i
Abstract	ii
致謝	iii
目錄	iv
第一章 緒論	1
1.1 研究動機	1
1.2 研究目的	2
1.3 論文組織架構	2
第二章 背景介紹與相關研究	4
2.1 大型語言模型	4
2.1.1 大型語言模型的原理與應用	4
2.1.2 Fine-tuning	5
2.1.3 In-context Learning	5
2.1.4 Chain of Thought	6
2.2 工具呼叫方法	6
2.2.1 結構化輸出	7
2.2.2 Python 程式碼執行	7
2.3 知識圖譜	8
2.3.1 圖資料庫與 Neo4j	8
2.3.2 節點 (Nodes)	9
2.3.3 關係 (Relationships)	9
2.3.4 屬性 (Properties)	9
2.4 相關研究	10
2.4.1 代理人的思維與規劃框架	10
2.4.2 工具的檢索與選擇	11
2.4.3 執行的穩健性與錯誤處理	12
2.4.4 專用框架與評估基準	13

第三章	APIKAgent 方法	14
3.1	APIKAgent 方法綜述	14
3.1.1	方法架構	14
3.1.2	步驟跳躍機制	15
3.1.3	Pass 機制：優化重新規劃效率	16
3.1.4	思維鏈	17
3.2	API 知識圖譜設計與檢索	17
3.2.1	API 知識圖譜設計	17
3.2.2	API 知識圖譜檢索演算法	19
3.2.3	API 知識圖譜於參數尋找的應用	22
3.3	APIKAgent 的任務規劃與執行	24
3.3.1	任務規劃	24
3.3.2	任務計畫執行與錯誤監控	25
3.3.3	重新規劃與 Pass 機制	26
第四章	APIKAgent 系統實作	29
4.1	知識圖譜構建	29
4.2	任務規劃模組	30
4.2.1	任務規劃流程	30
4.2.2	任務計畫的結構與內容	31
4.3	任務計畫執行模組	32
4.3.1	執行 API	32
4.3.2	資料處理	34
4.4	重新規劃模組	35
4.5	人為監督與介入機制	37
4.6	其他實作細節	38
4.6.1	提示詞模板	38
4.6.2	記憶管理	39
第五章	實驗與結果分析	40
5.1	研究問題	40

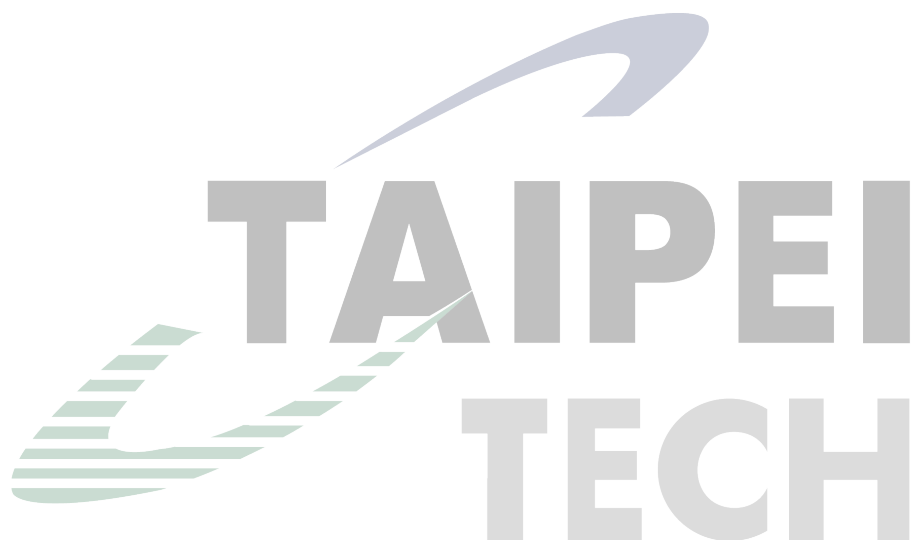
5.2	實驗設置	40
5.2.1	實驗基準	40
5.2.2	評估指標	40
5.2.3	實驗環境	41
5.3	實驗一	42
5.4	實驗二	46
5.5	實驗三	48
第六章	結論與未來研究方向	50
6.1	結論	50
6.2	未來研究方向	50
參考文獻		53
附錄		55
A	Prompts	55
A.1	計畫生成 Prompt	55
A.2	驗證計畫內容 Prompt	56
A.3	終止步驟分析結果	57
A.4	API 內部參數相依性分析 Prompt	58
A.5	可調整參數生成 Prompt	59
A.6	可調整參數生成 Prompt	60
A.7	驗證 API 執行結果 Prompt	62
A.8	驗證已有資訊是否能夠進行資料處理 Prompt	64
A.9	Python 程式碼生成 Prompt	64
A.10	驗證 python 執行結果 Prompt	65
A.11	重新規劃 Prompt	66
B	程式碼	68
B.1	自動構建知識圖譜程式碼	68

圖目錄

3.1	APIKGAgent 架構圖	15
3.2	API 知識圖譜示例	18
3.3	API 檢索範例	21
4.1	任務規劃流程圖	30
4.2	API 執行流程圖	33
4.3	資料處理流程圖	34
4.4	重新規劃流程圖	36
5.1	APIKGAgent 在 dev 資料集中的錯誤類型分佈	44
5.2	APIKGAgent 在 test_normal 資料集上各任務的重新規劃次數	45
5.3	APIKGAgent 在 test_normal 資料集上的使用 token 數量分佈 (a: 輸入, b: 輸出, c: 總數), 綠色三角形表示平均值。	46
5.4	APIKGAgent 在 test_normal 資料集上執行時間數量分佈, 綠色三角形表示平均值。	46
5.5	不同實驗設定下, 因達到重新規劃次數上限而失敗的任務數量	49

表目錄

3.1	API 知識圖譜節點類型	18
3.2	API 知識圖譜關係類型	18
5.1	實驗硬體與軟體環境	41
5.2	代理人設定	41
5.3	各方法於 AppWorld test_normal 資料集的表現	42
5.4	各組件消融對任務完成率之影響	47
5.5	思維鏈與思考模式對任務完成率的影響	48



第一章 緒論

1.1 研究動機

隨著大型語言模型（Large Language Model, LLM）的突破性發展，其強大的語言理解、推理與規劃能力，為實現能夠自主操作軟體服務的智慧代理人（Agent）帶來了新的契機。這些代理人透過調用應用程式介面（Application Programming Interface, API）與外部世界互動，被期望能自動化完成日益複雜的多步驟任務。

目前主要循兩條路徑來賦予 LLM 使用工具的能力。第一種路徑以「上下文學習」（In-Context Learning）為核心，代表性方法如 ReAct[1] 等，透過在提示（Prompt）中提供詳盡的 API 文件以及少量執行範例（Few-shot Examples），引導模型在執行階段動態生成操作指令。第二種路徑則是透過「微調」（Fine-tuning）[2] 技術，將特定工具的使用知識直接內化至模型參數中，使模型能更直接地生成 API 呼叫。

儘管上述方法已取得初步成功，但在面對真實世界中大規模、動態且相互依賴的 API 生態時，仍顯現出其固有的局限性：

- **上下文長度限制**：基於上下文學習的方法，其效能高度依賴於提示中所含資訊的品質與完整性。然而，LLM 的上下文視窗（Context Window）長度有限，當 API 數量龐大時，無法將所有文件完整納入，導致模型因資訊不全而難以選擇正確的 API。此一限制迫使開發者在 API 覆蓋廣度與文件詳細度之間做出困難的權衡。
- **微調成本與彈性問題**：微調方法雖然能提升模型在特定任務上的表現，卻也帶來了高昂的訓練成本與僵化問題。在現代軟體開發實踐中，API 經常迭代更新。每當 API 規格變更、新增或棄用時，重新微調模型不僅耗費大量運算資源與時間，也阻礙了系統的敏捷性與可擴充性。
- **規劃的可靠性與模型幻覺**：現有方法普遍缺乏對 API 之間複雜依賴關係的深刻理解。例如，執行某個 API 可能需要另一 API 的輸出作為其輸入參數。在缺乏結構化知識引導的情況下，LLM 容易產生「幻覺」（Hallucination），忽略此類前置條件，生成語法正確但邏輯錯誤的執行計畫，最終導致任務失敗。此問題在需要長鏈推理與嚴格執行順序的任務中尤為突出。

為應對上述挑戰，本論文認為，將 API 的領域知識從 LLM 的核心推理能力中分離，並以結構化的方式進行外部管理，是實現可擴充、高適應性代理人的關鍵。因此，本論文旨在探索一種無需微調的代理人框架，透過知識圖譜輔助，提升其在複雜 API 環境下的任務成功率與系統可靠性。

1.2 研究目的

本論文提出一個名為 APIKGAgent 的代理人框架，旨在解決現有基於大型語言模型的代理人在 API 調用方面所面臨的挑戰。本論文的核心目的如下：

1. **建構一個無需微調的代理人框架：**透過將 API 的知識外部化，儲存於知識圖譜（Knowledge Graph, KG）中，如要使用特定 API 時，再透過檢索對應的 API，使代理人無需經過昂貴的微調過程，即可理解並使用 API。
2. **實現 API 的動態管理與擴充：**藉由知識圖譜的輔助，系統可以輕易地新增、刪除或更新 API 資訊，框架能夠即時適應這些變動，大幅提升系統的擴充性與維護性。
3. **有效處理複雜的 API 依賴關係：**利用知識圖譜明確定義 API 之間的輸入輸出依賴，引導 LLM 進行正確的任務規劃，從而減少模型幻覺。
4. **整合人為監督與介入機制：**為確保任務執行的可靠性與安全性，本框架設計了人為監督機制，允許使用者在任務規劃等關鍵階段進行審查與調整，在自動化與人工監管之間取得平衡。

總體而言，本論文旨在開發一個更穩健、適應性更強的智慧代理人，使其能夠在真實世界中安全、有效地完成需要多步驟 API 調用的複雜任務。

1.3 論文組織架構

本論文共分為六章，其架構安排如下：

- **第一章緒論：**說明本論文的動機、目的與挑戰，並概述論文的整體架構。

- **第二章背景介紹與相關研究：**介紹與本論文相關的背景知識與相關研究，包括大型語言模型、知識圖譜與工具呼叫等。
- **第三章研究方法：**闡述本論文提出的 APIKGAgent 框架的理論基礎與核心設計，說明如何利用知識圖譜輔助任務規劃。
- **第四章系統實作：**詳細介紹 APIKGAgent 的各個模組，包括知識圖譜的自動化建構、任務規劃、計畫執行、重新規劃以及人為監督與介入機制。
- **第五章實驗與結果分析：**說明實驗環境、評估指標與方法，並展示與分析 APIKGAgent 在模擬測試環境中的表現。
- **第六章結論與未來研究方向：**總結本論文的研究貢獻，並提出未來可能的研究方向。



第二章 背景介紹與相關研究

2.1 大型語言模型

大型語言模型（Large Language Model, LLM）已成為推動人工智慧發展的關鍵技術。本節將探討 LLM 的核心概念。首先，2.1.1 小節將闡述 LLM 的基本原理及其多樣化的應用場景。隨後，我們將分別在 2.1.2 小節和 2.1.3 小節中介紹兩種使 LLM 獲取並應用特定領域知識的關鍵技術：Fine-tuning 與 In-context Learning。最後，2.1.4 小節將探討如何透過思維鏈（Chain of Thought）方法來增強 LLM 的推理能力。

2.1.1 大型語言模型的原理與應用

大型語言模型（Large Language Models, LLMs）是近年來人工智慧領域取得顯著突破的技術之一。這些模型通常基於 Transformer 架構 [3]，並透過在海量的文本資料上進行預訓練，使其能夠理解和生成類似人類的自然語言。LLM 的核心能力在於其對上下文的深刻理解以及生成連貫、相關文本的能力。

近年來，LLM 的應用進一步拓展到構建智慧代理人（Agent）的領域。透過將 LLM 作為代理人的操作核心，使其不僅能理解和生成文本，更能基於對環境的理解和任務目標，進行自主規劃、決策和執行動作。這種以 LLM 為核心的代理人展現出在複雜任務中進行推理、與外部工具交互以及適應動態環境的潛力，為自動化和智慧化開闢了新的可能性。

本小節旨在介紹大型語言模型的基本原理和其廣泛的應用潛力，包括其作為代理人核心的最新發展。接下來的內容將為後續探討 LLM 的進階技術，如領域知識的學習方法（2.1.2 小節的 Fine-tuning 與 2.1.3 小節的 In-context Learning）以及提升 LLM 推理能力的思維鏈法（2.1.4 小節的 Chain of Thought），奠定必要的基礎。透過對 LLM 原理的理解，研究者與開發者可以更有效地利用這些強大的工具來解決複雜問題並推動相關領域的創新。

2.1.2 Fine-tuning

Fine-tuning[2] 是一種調整預訓練大型語言模型以適應特定任務或領域知識的方法。這個過程通常涉及在一個較小的、特定任務的資料集上繼續訓練 LLM 的參數。透過 Fine-tuning，模型能夠學習到特定領域的術語、風格和知識，從而在該領域的任務上表現更佳。

然而，Fine-tuning 也存在一些挑戰。首先，它通常需要大量的標註資料集來進行有效的訓練 [4][5]，資料的收集與標註成本較高。其次，Fine-tuning 後的模型在一定程度上是靜態的，若需要讓模型理解並使用新的 API 或工具，往往需要重新進行 Fine-tuning，缺乏動態適應性，這導致維護和更新的成本增加。考量到這些因素，特別是對於需要頻繁更新可用工具或 API 的場景，Fine-tuning 可能不是最具成本效益的選擇。因此，本論文的研究方法並未採用 Fine-tuning。

2.1.3 In-context Learning

In-context Learning (ICL)[6] 是另一種讓大型語言模型適應特定任務或情境的方法，其核心思想是透過在模型的輸入提示（prompt）中提供少量範例（few-shot）或任務描述，來引導模型在不更新其內部參數的情況下執行任務。LLM 會利用其在預訓練階段學習到的模式識別能力，從輸入的上下文範例中理解任務要求，並生成相應的輸出。

相較於 Fine-tuning，In-context Learning 具有無需額外訓練、能夠快速適應新任務的優點。然而，ICL 也面臨其固有的限制，其中最主要的是輸入上下文長度的限制（input context limitation）。現有的 LLM 對於輸入提示的長度通常設有上限，如果需要讓 LLM 理解並選擇使用的 API 數量非常龐大，將所有 API 的詳細描述都放入輸入上下文中可能會超出此限制，導致模型無法有效處理或忽略部分重要資訊，進而影響其決策和工具選擇的準確性。

為了解決 In-context Learning 在面對大量 API 時的理解困難，本論文提出了一種基於知識圖譜的方法。透過將可用的 API 結構化為知識圖譜，我們可以根據當前任務需求，動態地從知識圖譜中檢索並篩選出相關的 API 子集。如此一來，僅需將這個經過限縮的、高度相關的 API 資訊提供給 LLM 作為輸入上下文，既能有效利用 In-context Learning 的優勢，又能避免超出上下文長度限制的問題。

2.1.4 Chain of Thought

思維鏈 (Chain of Thought, CoT) [7] prompting 是一種提升大型語言模型推理能力的技術。其核心思想是引導 LLM 在回答複雜問題時，不僅僅給出最終答案，而是先產生一系列中間的、連貫的思考步驟，模擬人類解決問題時的推理過程。透過明確地展示思維鏈，LLM 能夠更有效地分解問題、進行邏輯推演，並最終得到更準確、更可靠的結果。

在傳統的 prompting 方式中，LLM 可能直接跳到結論，尤其是在面對需要多步驟推理的任務時，容易出錯。而 CoT prompting 則鼓勵模型「思考地更慢、更深入」。研究表明 [7]，即使是沒有經過專門 CoT 訓練的 LLM，在被提示生成思考步驟後，其在算術、常識和符號推理等任務上的表現也能得到顯著提升。

本論文在設計與 LLM 的交互提示 (prompt) 時，也借鑒了 Chain of Thought 的理念。我們透過精心設計的 prompt 結構，引導 LLM 進行一步一步的思考，例如在進行參數判斷時，會提示 LLM 首先分析任務需求，然後列出可能的參數，接著評估每個參數應填入的值，最終做出決策。這種循序漸進的思考方式有助於 LLM 更全面地理解任務上下文，減少錯誤的產生，從而獲得更佳的答案與執行結果。

2.2 工具呼叫方法

隨著 LLM 能力的持續增強，使其能夠有效地與外部世界互動並自主執行日益複雜的任務，已成為人工智慧領域一個重要的研究方向。為了擴展 LLM 的應用邊界，研究者們致力於發展各種方法，使 LLM 能夠利用預先定義的工具或應用程式介面 (Application Programming Interfaces, APIs) 來獲取即時資訊、執行特定計算或操作外部系統。本節旨在探討當前大型語言模型中幾種主流的工具呼叫機制，並藉此為後續章節詳細闡述本論文所提出的 APIKGAgent 方法 (詳見第三章) 中，如何實現工具的智慧選擇、調用與管理，提供必要的背景知識與理論基礎。

2.2.1 結構化輸出

結構化輸出是 LLM 用以呼叫外部工具的一種核心方法。其主要思想是引導 LLM 生成特定格式的文本（常見格式如 JSON、XML 或 YAML），該文本明確指定了所需調用的工具名稱及其必要的參數。例如，當 LLM 判斷需要查詢天氣資訊時，它可以生成類似以下的 JSON 對象：

```
{
  "tool_name": "get_weather",
  "parameters": {
    "location": "Paris",
    "date": "2025-06-17"
  }
}
```

當代理人（Agent）在接收到此類結構化輸出後，會對其進行解析，並根據指令內容實際執行相應的工具（如上例中的「get_weather」）。工具執行完畢後，其結果將被回傳給 LLM，作為其後續對話生成或任務推進的依據。此外，通過預先定義工具的描述檔案（Schema），可以有效地約束 LLM 的輸出行為，從而提高工具調用的準確性。目前許多主流的 LLM 應用框架，例如 LangChain [8] 與 OpenAI 所提供的 Function Calling 功能 [9]，均廣泛採用了此類機制，允許開發者定義工具介面，並使 LLM 能夠以結構化的方式請求工具調用。在本論文所提出的 APIKGAgent 框架（詳見第三章）中，LLM 與系統的交互，亦遵循結構化輸出的原則。

2.2.2 Python 程式碼執行

除了前述的結構化輸出方法外，大型語言模型進一步具備了直接生成並執行程式碼的能力，藉此實現更為靈活的工具運用與複雜任務處理。一個典型的例子是 OpenAI 的 Code Interpreter 功能 [10]，該功能允許 LLM 根據使用者需求，動態編寫並執行 Python 程式碼片段，以完成諸如數據分析、圖表繪製、文件格式轉換等多樣化操作。

在此模式下，LLM 會生成一段包含特定函式庫、函式調用及數據處理邏輯的 Python 腳本。隨後，這段腳本將在一個受控且安全的執行環境（通常是一個經過安全加固的沙盒 (sand box) 化 Python 解釋器）中運行，其執行結果（例如文本輸出、生成的圖像或潛在的錯誤訊息）會被回傳給 LLM，以供其進一步分析或呈現。

這種方法的優勢在於其高度的靈活性和表達能力，LLM 可以直接利用 Python 生態中豐富的庫來完成複雜任務，而無需為每個細微功能都預先定義一個結構化的工具。然而，直接執行 LLM 生成的程式碼也帶來了顯著的安全風險，需要嚴格的沙盒環境來防止惡意程式碼的執行。此外，確保 LLM 生成正確且高效的程式碼也是一個挑戰。

本論文的方法雖然不直接執行由 LLM 動態生成的通用 Python 程式碼片段來呼叫外部 API，但在其資料處理步驟（詳見 4.3.2）中，涉及數據轉換、計算與整合的步驟，仍會使用此方法進行處理資料。

2.3 知識圖譜

知識圖譜（Knowledge Graph, KG）作為一種結構化的知識表示方法，近年來在資訊檢索、推薦系統、自然語言處理等多個領域展現出巨大的潛力。它以圖的形式對現實世界中的實體、概念及其之間的複雜關係進行建模，從而為機器提供了一種更接近人類認知方式的知識組織框架。本節將深入探討知識圖譜的核心構成要素，為後續章節中本論文提出的 APIKGAgent 方法（詳見第三章）奠定理論基礎。首先，2.3.1 小節將介紹圖資料庫的基本概念，並說明為何選擇 Neo4j 作為本論文的技术實現。接著，2.3.2、2.3.3 與 2.3.4 小節將分別詳細闡述知識圖譜的三大核心元素：節點、關係與屬性，並解釋它們如何共同作用以構建一個豐富且可查詢的知識網絡。

2.3.1 圖資料庫與 Neo4j

知識圖譜的實現高度依賴於專為儲存與查詢圖結構數據而設計的圖資料庫。市面上有數種主流的圖資料庫，例如 Amazon Neptune、JanusGraph 等，它們各自具備不同的特性與應用場景。然而，在眾多選項中，Neo4j 以其成熟的社群、豐富的文件以及原生的圖處理能力脫穎而出，成為學術界與業界建構知識圖譜時常用的解決方案之一。

2.3.2 節點 (Nodes)

在知識圖譜中，節點是表示實體或概念的基本單元。它們是圖中的頂點，代表了我們關心的任何對象。在本論文的 API 知識圖譜中，節點可以代表一個具體的 API（例如 'getUserProfile'）、一個功能性的關鍵字（例如「使用者」、「查詢」），或是一個 API 的參數（例如 'userID'）。

每個節點都可以擁有一或多個標籤（Label），用以對節點進行分類。標籤提供了一種為節點分組的機制，使得查詢特定類型的所有節點變得高效。例如，所有代表 API 的節點都可以被賦予 'API' 標籤。

2.3.3 關係 (Relationships)

如果說節點是知識圖譜中的「名詞」，那麼關係就是連接這些名詞的「動詞」。關係是連接兩個節點的有向邊，用以描述它們之間的具體關聯。每一條關係都必須有一個類型（Type），用以定義其語義。例如，一條從 'getUserProfile' API 節點指向 'userID' 參數節點的關係，其類型可以是 'REQUIRE_PARAMETER'，明確地表達了「需要參數」的意涵。關係的方向性對於精確表達知識至關重要。'A-[KNOWS]->B' 與 'B-[KNOWS]->A' 可能代表了不同的意義。在 API 知識圖譜中，關係的方向清楚地定義了 API 之間的依賴與數據流動方向。

2.3.4 屬性 (Properties)

屬性是以鍵值對（Key-Value Pairs）形式儲存在節點和關係上的詳細資訊。它們為知識圖譜中的實體和關聯提供了豐富的上下文描述。例如，一個 API 節點除了有 'API' 標籤外，還可以包含多個屬性，如 'name': 'getUserProfile'、'description': '獲取使用者個人資料' 以及 'method': 'GET'。同樣地，關係也可以擁有屬性，例如，一條表示 API 呼叫依賴的關係可以有一個 'priority' 屬性來標示其執行的優先級。

透過節點、關係與屬性的結合，屬性圖模型能夠以極高的靈活性和豐富的細節來描繪複雜的知識結構。這種表達能力是本論文選擇使用知識圖譜來管理和推理 API 相關知識的根本原因。

2.4 相關研究

大型語言模型（LLM）的快速發展，推動了能夠自主規劃並執行複雜任務的智慧代理人（LLM Agent）的興起。這些代理的核心能力之一，在於有效利用外部工具與 API 來擴展其知識邊界和行動能力。本章旨在回顧並分析大型語言模型在工具使用領域的關鍵研究，從代理思維框架、核心的工具檢索與選擇技術，到執行過程中的錯誤修正與重新規劃策略，最後探討專用框架與評估基準。

2.4.1 代理人的思維與規劃框架

早期的大型語言模型代理主要依賴於模型內建的知識進行推理，但很快研究者便發現，為了應對真實世界的複雜性，代理需要一套清晰的「思考-行動」模式。

Yao 等人（2022）提出的 **ReAct (Reasoning and Acting)** 框架是此領域的開創性工作 [1]。ReAct 的核心思想是將「推理（Thought）」與「行動（Action）」交錯進行，形成一個「思考-行動-觀察」的迭代循環。模型首先生成解決問題的思路，然後決定執行一個具體的工具（行動），接著根據工具返回的觀察結果（Observation）來修正下一步的思考與行動。這種方式賦予了模型極佳的適應性，使其能夠在執行過程中動態調整策略。然而，ReAct 這種即時反應的模式，對於需要長遠規劃的複雜任務，可能因缺乏全域視野而陷入局部最優或不必要的重複探索。

為了解決此問題，後續研究提出將規劃與執行階段明確分離的 **Plan-and-Execute**[11] 框架。此框架的核心特徵是將任務處理劃分為兩個獨立階段：「規劃」（Planning）和「執行」（Execution）。在規劃階段，代理會通盤思考，一次性生成完整的任務步驟；隨後在執行階段，嚴格按照既定計畫進行操作。這種模式的優點在於，透過預先的周詳規劃，能夠提升任務執行的連貫性與效率，降低因反覆試錯帶來的成本。但其缺點也相當明顯：由於計畫的僵化，一旦執行過程中遭遇預期外的失敗或環境變化，整個計畫可能失效，缺乏 ReAct 的即時適應能力。

在此基礎上，Xu 等人（2023）提出了 **ReWOO (Reasoning WithOut Observation)**，一個高效的模組化框架 [12]。ReWOO 將代理的角色精細劃分為 Planner、Worker 與 Solver。Planner 負責生成完整的推理計畫，但與 Plan-and-Execute 不同的是，計畫中的

每一步都明確標示了需要調用的工具。Worker 則專注於並行執行這些工具調用，收集所有必要的資訊。最後，Solver 根據原始計畫和 Worker 收集到的所有資訊，生成最終答案。ReWOO 的創新之處在於將昂貴的 LLM 推理過程（規劃與解答生成）與相對廉價的工具調用過程解耦，提升了執行效率。

本論文提出的 APIKAgent 吸收了上述框架的優點。它類似於 Plan-and-Execute 和 ReWOO，採用了規劃與執行分離的架構以確保任務的整體連貫性。但與此同時，APIKAgent 透過後續將詳述的重新規劃機制，引入了類似 ReAct 的適應性，使其在面對執行失敗時能夠動態調整，而非僵化地終止，從而在規劃的穩定性與執行的靈活性之間取得了平衡。

2.4.2 工具的檢索與選擇

如何讓 LLM 掌握使用工具的能力，主要分為兩大技術路徑：基於情境學習（In-Context Learning）的檢索與基於模型微調（Fine-tuning）的學習。前者無需修改模型，僅在提示（Prompt）中提供工具資訊；後者則透過在大量的工具使用數據集上訓練模型，將工具使用能力內化到模型參數中。此類方法的核心挑戰在於，當可用的 API 或工具數量龐大時，如何準確、高效地為當前任務找到最合適的工具，並將其資訊有效地呈現給 LLM。傳統方法多依賴於對工具描述文檔的語義相似度搜索，但這種方式往往忽略了工具間的內在關聯。為此，ToolNet[13] 提出了一種基於工具圖的 API 選擇方法。ToolNet 將 API 視為圖中的節點，並利用邊的權重來學習 API 之間的協同關係與排斥關係，從而引導 LLM 從大量的 API 中選擇正確的 API，避免掉一些因為過時而無法使用的 API，並增強 LLM 對 API 依賴關係的理解。同樣為了解決大規模工具檢索問題，Anytool[14] 提出了一種分層式檢索方法，先通過高效的索引篩選出一個較小的候選工具集，再利用 LLM 進行精細選擇，有效應對了工具庫規模龐大的挑戰。另一條重要的研究路徑是將結構化知識與 LLM 的推理相結合。Think on Graph (ToG)[15] 便是一個典範。ToG 強調讓 LLM 在知識圖譜上進行思考，利用知識圖譜中實體與關係的結構化信息來引導和約束 LLM 的推理過程，從而實現更有效的知識檢索與多步推理。

與情境學習不同，微調方法旨在將工具使用能力直接注入模型本身。其中，ToolLLM (Qin et al., 2023)[4] 是此方向的代表性工作。ToolLLM 的研究團隊構建了一個名為 ToolBench 的大規模、高品質的工具使用指令微調數據集。他們提出了一種能夠自

動化地生成複雜且具備多步依賴關係的工具調用路徑作為模型的訓練樣本。經過微調的 ToolLLM 在處理複雜的工具序列任務上展現了極高的準確性。然而，微調方法的優勢同時也帶來了其根本性的局限：缺乏靈活性。模型的能力被「固化」在其訓練數據集所包含的工具集上。若要新增一個 API，就需要重新生成相關的訓練數據並對模型進行昂貴的再訓練，這在需要頻繁增刪或更換工具的動態真實世界場景中，顯得不切實際。

本論文的尋找 API 機制深受 ToG 方法的啟發，選擇了基於情境學習的檢索路徑以確保靈活性。我們不僅利用知識圖譜來表示 API，更為其設計了專門的節點與關係，將 API 的功能、參數需求與產出明確地結構化。相較於 ToolNet 使用邊的權重學習隱含關係，APIKGAgent 的知識圖譜提供了更為明確、可解釋的語義關聯，尤其在後續處理參數依賴、解決執行失敗等問題時，發揮了關鍵作用。

2.4.3 執行的穩健性與錯誤處理

在實際執行任務計畫時，失敗是常態而非例外，可能源於 API 呼叫失敗、參數錯誤、回傳結果不符預期等多種原因。因此，如何有效處理執行失敗，是衡量代理智慧程度的關鍵指標。

Tool-Planner [16] 採用一個兩階段的強化學習框架，其核心在於第二階段引入了一套獨特的雙重回饋機制，它不僅評估模型是否成功「完成任務」並提供有效答案，還會檢驗模型是否嚴格「遵循指令」，也就是有沒有使用使用者在提示中明確指定的工具、API 或類別。模型會根據這兩項指標計算出獎勵分數進行訓練，從而學會優先選擇那些既能有效解決問題、又能精準遵循使用者限制的思考路徑，從而提升了模型的可靠性與精確度。

LOOP [17] 提出了強化學習方法。該方法將任務互動建模為部分可觀測馬可夫決策過程 (POMDP)，並僅以簡單的任務完成回饋作為學習信號，讓模型自主學會查閱 API 文件、避免對 API 功能做出錯誤假設，從而提升在複雜應用環境中的任務完成率。

此外，**Intercode** [18]，設計了一個可控的 Python 執行環境。Intercode 允許 LLM 在一個受限的沙盒環境中執行 Python 程式碼，並且能夠回饋沙盒執行結果給 LLM。這種方法不僅提高了執行的安全性，還能夠讓 LLM 在執行過程中動態調整其行為，從而更好地適應環境變化。

APIKGAgent 的錯誤處理機制融合了多種思想。當 API 執行失敗時，特別

是因缺少必要參數而失敗時，系統會利用知識圖譜中的 REQUIRE_PARAMETER 和 RESPONSE_PARAMETER 關係，查找能夠生成該缺失參數的其他 API，並觸發重新規劃。這可以視為一種基於知識圖譜依賴分析的恢復策略。在資料處理方面，APIKGAgent 亦採用在受控環境中執行代碼的模式，確保了執行的安全性。

2.4.4 專用框架與評估基準

隨著研究的深入，社群也發展出針對特定場景的專用框架和標準化的評估基準。例如，**RestGPT** [19] 專注於 RESTful API 的調用，能夠自動解析 OpenAPI 文件並生成調用程式碼，在處理複雜的 Web API 任務時表現出色。而 **AppWorld** [20] 則是一個重要的評估基準。AppWorld 提供了一個包含多種應用的模擬環境，讓研究者可以在不直接與真實線上服務互動的情況下，評估 LLM 代理在多工具協同、跨應用操作等任務上的表現，極大地促進了領域內方法的可比較性與可重複性。



第三章 APIKGAgent 方法

本章將深入探討 APIKGAgent 的核心方法，詳細解析其系統架構、關鍵機制以及各組成部分的功能定位。首先，3.1 節將概述 APIKGAgent 的設計目標與整體架構，並介紹其主要組成部分。接著，3.2 節將說明 APIKGAgent 如何設計並應用知識圖譜。最後，3.3 節將敘述任務規劃與執行的流程與方法。

3.1 APIKGAgent 方法綜述

APIKGAgent 的設計目標在於賦予代理人 (Agent) 自主解析任務、整合並運用多樣化 API 的能力，從而自動化完成複雜的資訊處理與服務操作任務。為此，本論文提出一套無需微調的代理人框架，涵蓋 API 知識圖譜的構建、任務導向的 API 檢索、動態任務規劃、計畫執行，以及錯誤處理與重新規劃機制。

首先，3.1.1 節將宏觀介紹 APIKGAgent 的整體架構，闡明其主要構成要素與協同工作流程。隨後，3.1.2 節將聚焦於「步驟跳躍機制」，此機制賦予代理人動態調整執行路徑的能力，以應對包含循環或條件分支的複雜任務。緊接著，3.1.3 節將探討「Pass 機制」，此機制旨在透過複用先前已成功的執行結果，優化重新規劃後的效率。而 3.1.4 節將說明本方法是如何應用思維鏈 (Chain of thought, CoT) 來提升 LLM 的輸出品質。

3.1.1 方法架構

APIKGAgent 的整體架構如圖 3.1 所示。此架構的設計目標是實現一個無需微調 (Fine-tuning)、可靈活擴展 API 庫，並具備高度自我調整與執行的 API 操作框架。此架構主要由四大核心功能構成：API 檢索、任務規劃、計畫步驟執行以及重新規劃。

當系統接收到使用者下達的任務指令後，API 檢索功能首先啟動。其核心職責是基於任務的語義理解，從預先構建的 API 知識圖譜檢索出一組與當前任務目標高度相關的候選 API 集合。此過程不僅考慮 API 的功能描述，亦整合了 API 的輸入輸出參數、使用限制等多維度資訊。

隨後，篩選出的候選 API 集合將被傳遞至任務規劃階段。此階段作為系統的決策

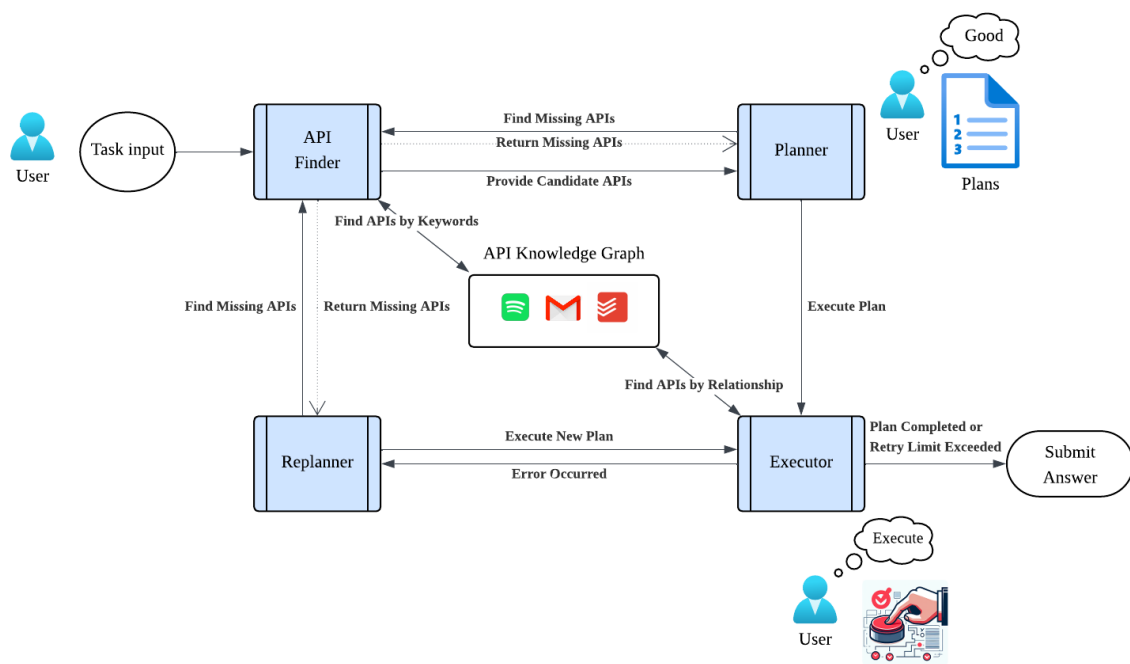


圖 3.1: APIKGAgent 架構圖

核心，負責根據任務的具體需求與 API 的特性，運用大型語言模型 (LLM) 生成一個結構化、邏輯清晰且可執行的多步驟任務計畫。此計畫詳細定義了 API 的調用順序、參數傳遞路徑以及必要的資料處理環節。

接下來，計畫步驟執行階段將依照任務計畫，按部就班地執行每一個步驟，包含實際的 API 呼叫或資料轉換等。在此過程中，系統會持續監控每一步的狀態與結果。

最後，若在執行過程中遭遇任何預期之外的狀況——例如 API 呼叫失敗、返回結果不符預期，或外部環境變化導致原計畫不再適用——重新規劃功能將被觸發。系統會分析失敗原因，並根據最新情境動態調整計畫，可能涉及 API 的替換或整體任務流程的重構。此一「感知-規劃-行動-調整」的循環流程，賦予了 APIKGAgent 適應真實世界複雜多變場景的關鍵能力。

3.1.2 步驟跳躍機制

傳統的線性任務執行流程，在處理涉及循環、條件判斷等複雜邏輯時，往往顯得能力不足。為此，本論文引入了步驟跳躍機制，旨在賦予 APIKGAgent 更高級別的執行彈性，使其能夠根據執行過程中的即時狀態和預設條件，動態地調整執行路徑。例

如，在一個「持續監控某項指標，直到其達到特定閾值後再執行下一步操作」的任務中，代理人需要重複呼叫查詢該指標的 API，並根據回傳結果判斷應繼續監控或跳轉至後續步驟。此機制使 APIKGAgent 得以有效處理此類非線性執行的任務，從而顯著拓寬其應用場景。

3.1.3 Pass 機制：優化重新規劃效率

在 APIKGAgent 的運作過程中，當執行遇到障礙或外部條件變化時，重新規劃機制會被觸發以生成新的任務計畫。Pass 機制專為此情境設計，其核心目標為：在新計畫生成後，識別出前一計畫中已成功執行且結果依然有效的步驟，從而跳過這些步驟，避免不必要的重複執行。此機制僅在重新規劃發生後介入。

當一個新的任務計畫生成後，Pass 機制會對比新計畫中的步驟與前一次規劃的執行歷史。其運作原理主要包含以下面向：

- **歷史執行記錄查閱：**APIKGAgent 維護前次規劃的執行歷史，其中包含每個已執行步驟的詳細資訊（如所呼叫的 API、使用的參數、產生的結果等）。
- **步驟等效性與結果有效性判斷：**對於新計畫中的每一個步驟，Pass 機制會檢查是否存在一個與之等效且已成功執行的歷史步驟。等效性判斷不僅基於步驟的名稱或類型，更會綜合考量其上下文、目標與依賴關係。
- **結果複用與執行跳過：**若找到等效的歷史步驟，APIKGAgent 將直接複用該歷史結果，並跳過該步驟在新計畫中的實際執行。此舉不僅避免了重複的 API 呼叫或資料處理，更對於會修改外部狀態的 API（如新增、刪除操作），可防止因重新規劃導致的非預期重複操作。

透過 Pass 機制，APIKGAgent 在重新規劃後能夠更快速地銜接至新的或未完成的任務環節，提升系統的反應速度和整體執行效率，同時也保障了操作的一致性與系統狀態的穩定性。

3.1.4 思維鏈

思維鏈 (Chain of Thought, CoT) 是一種經研究證實 [7] 能顯著提升大型語言模型 (LLM) 於複雜任務中推理能力的方法。其核心在於引導 LLM 進行分步推理，要求其在產出最終結果前，先闡述其思考過程與決策依據。本研究將此方法應用於各階段的提示詞 (Prompt) 設計中，透過預定義的步驟，以引導 LLM 進行逐步思考。而針對判斷性任務，除了要求 LLM 列出分析步驟外，更會要求其提供最終結論的具體理由，藉此提升決策過程的透明度與結果的可信度。

以任務規劃階段為例，本論文設計的提示詞會引導 LLM 依循以下步驟：首先，深入分析任務的具體需求；其次，檢視並評估候選 API 集合中各 API 的功能與適用性；最後，綜合前述分析，生成一個結構化的任務計畫。此一設計確保 LLM 在生成計畫前，已對任務目標與可用資源進行了系統性的考量，從而提升最終產出計畫的品質。

詳細的提示詞設計原則與模板將於4.6.1節說明，而各階段所使用的完整提示詞內容則收錄於附錄A。

3.2 API 知識圖譜設計與檢索

在前述章節對 APIKGAgent 的宏觀架構與核心機制進行概述後，本節將深入探討其知識核心——API 知識圖譜的設計與應用。API 知識圖譜不僅是 API 的儲存庫，更是實現智慧化檢索與推理的基礎。本節將首先介紹其設計原則，隨後詳細說明如何利用此圖譜進行 API 檢索與參數尋找。

3.2.1 API 知識圖譜設計

API 知識圖譜的設計，旨在以結構化的方式整合 API 資訊，為後續的自動化檢索與推理提供知識基礎。APIKGAgent 的知識圖譜在節點與關係的設計上，如表 3.1 與表 3.2 所示。

透過這樣的節點與關係設計，APIKGAgent 能夠有效解析知識圖譜中的結構化資訊，並以此為基礎進行 API 檢索、參數尋找等應用，最終如圖3.2所示。詳細的知識圖譜構建流程將在後續4.1節中進一步說明。

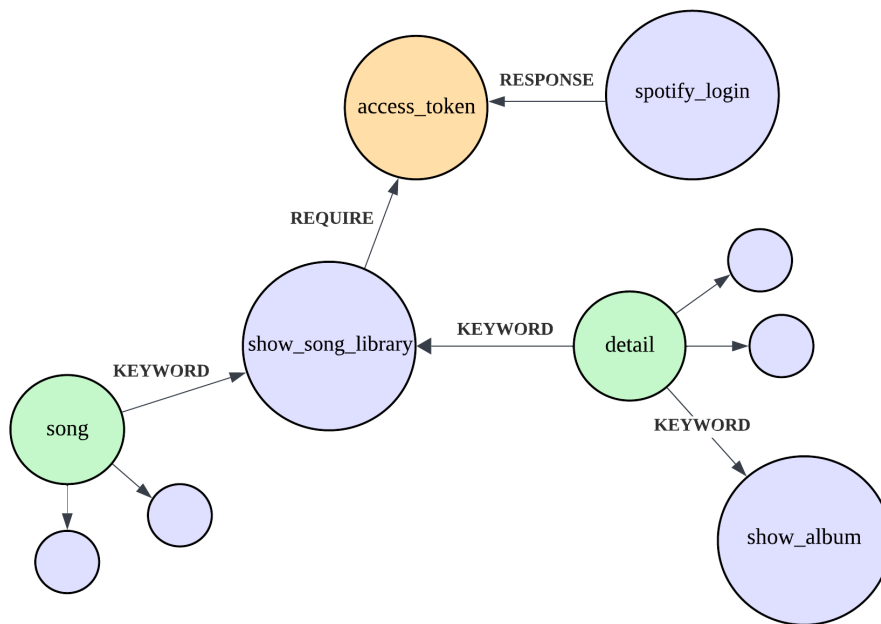


圖 3.2: API 知識圖譜示例

表 3.1: API 知識圖譜節點類型

節點類型	說明
API 節點	代表系統中所有可用的 API，記錄名稱、功能描述等。可細分為「搜尋型」與「執行型」。
參數節點	描述 API 所需的各項參數，包括名稱、資料型態、約束條件等。
關鍵字節點	標示 API 的功能特性或應用領域，與 API 節點建立語意關聯。

表 3.2: API 知識圖譜關係類型

關係類型	說明
REQUIRE_PARAMETER	連結 API 節點與其所需的參數節點，標示 API 的輸入。
RESPONSE_PARAMETER	連結 API 或資料節點與其相關的參數節點，標示 API 執行後產生的參數。
KEYWORD_PARAMETER	連結 API 節點與關鍵字節點，表示 API 與功能特性的關聯性。

3.2.2 API 知識圖譜檢索演算法

本節旨在詳細闡述 APIKGAgent 如何從龐雜的 API 集合中，萃取與當前任務高度相關的 API。此部分設計受 Think on Graph (ToG) [15] 方法啟發。ToG 強調大型語言模型 (LLM) 與知識圖譜的深度融合，以提升萃取資訊的能力。本論文借鑒此思想，使 APIKGAgent 能夠有效利用知識圖譜的結構化信息與 LLM 的語義理解能力，實現 API 檢索。

API 檢索的核心運作流程如演算法 1 所示，其目標是透過一系列迭代步驟，從知識圖譜中識別並篩選出最符合任務需求的 API。

Algorithm 1: API 知識圖譜檢索演算法

Input: 使用者任務 T
Output: 一組候選 API C_{final}

```
1  $K_{pool} \leftarrow \text{LLM.ExtractKeywords}(T);$ 
2  $C_{candidate} \leftarrow \emptyset;$ 
3 while true do
4    $K_{ranked} \leftarrow \text{LLM.RankKeywords}(K_{pool}, T);$ 
5    $K_{filtered} \leftarrow \text{FilterByThreshold}(K_{ranked});$ 
6    $C_{matched} \leftarrow \text{KG.SearchAPIsByKeywords}(K_{filtered});$ 
7    $C_{ranked} \leftarrow \text{LLM.RankAPIs}(C_{matched}, T);$ 
8    $C_{candidate} \leftarrow \text{Distinct}(C_{candidate} \cup C_{ranked});$ 
9    $is\_sufficient \leftarrow \text{LLM.CheckAPISufficiency}(C_{candidate}, T);$ 
10  if  $is\_sufficient$  or  $ReachedMaxIteration$  then
11    break;
12   $K_{from\_apis} \leftarrow \text{KG.ExtractKeywordsFromAPIs}(C_{candidate});$ 
13   $K_{new} \leftarrow K_{from\_apis} \setminus K_{pool};$ 
14  if  $K_{new}$  is empty then
15    break;
16   $K_{pool} \leftarrow K_{pool} \cup K_{new};$ 
17  $C_{final} \leftarrow C_{candidate};$ 
18 return  $C_{final};$ 
```

演算法 1 的詳細步驟說明如下：

1. **初始關鍵詞提取:** 首先，利用大型語言模型從使用者輸入的任務 T 中提取一組初始關鍵詞，形成關鍵詞池 K_{pool} 。此步驟旨在捕捉任務的核心語義概念。
2. **迭代式 API 檢索與擴展:** 接下來，系統進入迭代循環，直至尋獲足夠的 API 或滿足終止條件。

- **關鍵詞評分與排序:** 在每次迭代中，LLM 根據與任務 T 的相關性，對關鍵詞池 K_{pool} 中的關鍵詞進行評分與排序，生成列表 K_{ranked} 。
 - **關鍵詞過濾:** 根據預設的相關性閾值，篩選 K_{ranked} ，生成一組高度相關的關鍵詞 $K_{filtered}$ ，以提升後續檢索的精準度。
 - **API 匹配:** 利用 $K_{filtered}$ ，系統在 API 知識圖譜中搜尋具有對應關鍵字的 API，形成候選集合 $C_{matched}$ 。
 - **API 評分與排序:** LLM 根據 API 功能與任務 T 的契合度，對 $C_{matched}$ 中的 API 進行評分與排序，生成列表 C_{ranked} 。
 - **候選集更新與去重:** 將 C_{ranked} 中評分最高的 API 併入候選 API 總集 $C_{candidate}$ ，並確保集合中無重複項。
 - **API 充分性檢查:** LLM 評估當前的 $C_{candidate}$ 是否足以完整執行任務 T 。若判斷為是，或已達最大迭代次數，則終止循環。
 - **關鍵詞擴展:** 若 API 尚不充分，則從 $C_{candidate}$ 中的 API 反向提取新的關聯關鍵詞 K_{from_apis} ，並將其中未出現於 K_{pool} 的新關鍵詞 K_{new} 篩出。
 - **關鍵詞池更新:** 若 K_{new} 非空，則將其加入 K_{pool} ，以擴大下一輪的檢索範圍；反之，若無新關鍵詞可擴展，則終止循環。
3. **最終候選 API 集合:** 迴圈結束後，系統返回 $C_{candidate}$ 作為最終的候選 API 集合 C_{final} ，以供後續的任務規劃與執行。

API 檢索範例

為了具體說明演算法 1 的運作方式，本節將以一個實際任務作為範例進行闡述，如圖 3.3 所示。此範例將作為後續章節的基礎，以展現 APIKGAgent 的完整運作流程。圖中綠色節點為關鍵字節點，藍色節點為 API 節點，而橘色節點為參數節點。

1. **輸入任務 (T):** "What is the title of the most liked song in my Spotify song library?"
2. **關鍵字萃取:** LLM 從任務描述中萃取出關鍵字，形成關鍵字池 $K_{pool} = \{\text{title, like, song, spotify, library}\}$ 。

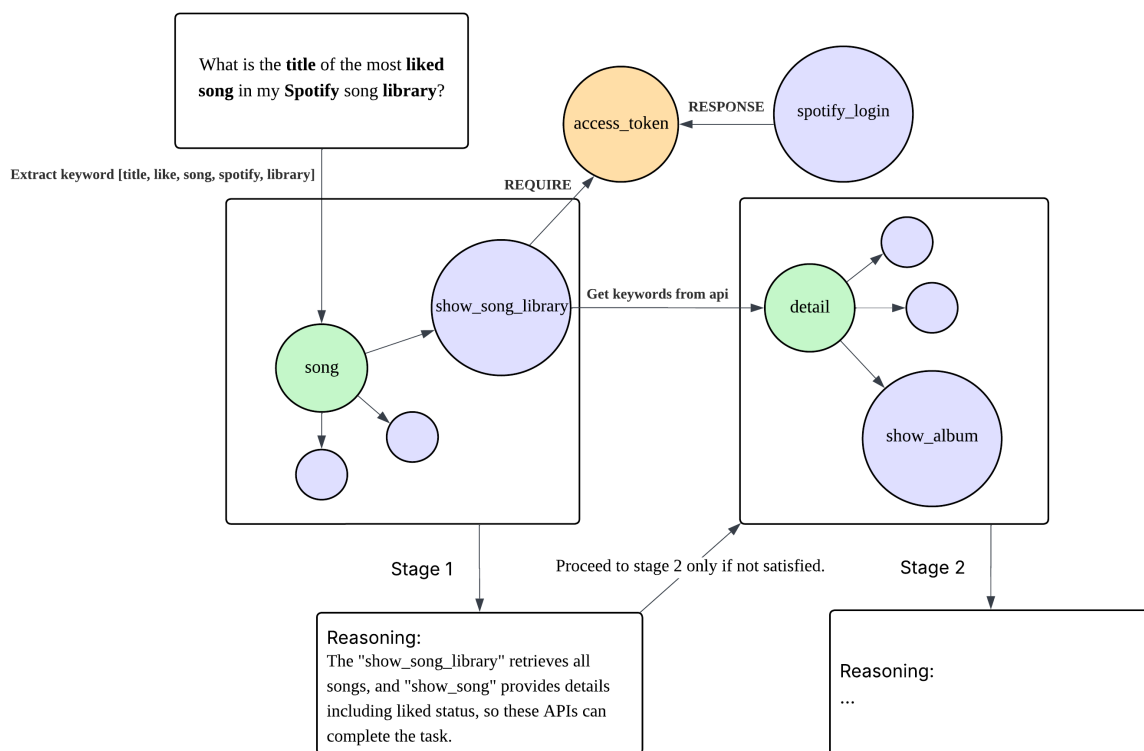


圖 3.3: API 檢索範例

3. **關鍵字評分與篩選:** LLM 對關鍵字進行相關性評分 (滿分 10 分)，並設定閾值為 6 分。

- song: 10 分
- spotify: 10 分
- library: 10 分
- like: 9 分
- title: 8 分

所有關鍵字的評分均高於閾值，因此全部保留於 $K_{filtered}$ 。

4. **API 匹配與評分:** 系統使用 $K_{filtered}$ 在知識圖譜中搜尋，並由 LLM 對匹配到的 API 進行評分。

- play_music: 0 分
- show_song_library: 10 分

- show_current_song: 1 分
- show_song: 9 分
- ...

5. **候選 API 篩選:** 評分高於閾值 6 分的 API 被加入候選集合

$C_{candidate} = \{\text{show_song_library}, \text{show_song}\}$ 。

6. **API 充分性檢查與關鍵字擴展:** LLM 判斷當前的 $C_{candidate}$ 是否足以完成任務。

LLM 判斷結果：足夠完成任務。判斷理由為「show_song_library 可取得使用者音樂庫中的所有歌曲，而 show_song 可取得每首歌曲的詳細資訊，其中包括按讚狀態，因此組合這兩個 API 已能滿足任務需求。」

由於 LLM 判定當前 API 集合已足夠，迭代循環終止。若判定不充分，則會從排名最高的 API (show_song_library, show_song) 反向提取關聯的關鍵字，並將新關鍵字加入 K_{pool} 進行下一輪迭代。

7. **最終輸出:** 演算法返回最終的候選 API 集合 $C_{final} = \{\text{show_song_library}, \text{show_song}, \dots\}$ ，以供後續任務規劃階段使用。

3.2.3 API 知識圖譜於參數尋找的應用

在 API 調用過程中，大型語言模型 (LLM) 可能因未能完全理解參數的語義脈絡或預期來源，產生「幻覺」(Hallucination) 現象，即臆測並填入不正確的參數值，從而導致 API 執行失敗。為解決此問題，APIKGAgent 設計了結合 API 知識圖譜的參數尋找機制，旨在引導 LLM 深入思考每個參數的具體意義及其資料來源，從而降低幻覺現象的發生率。

API 知識圖譜將結構化的 API 資訊有系統地組織起來，為 LLM 提供明確且可追溯的推理依據。當某一 API 需要被執行，但其必要的輸入參數值未知時，APIKGAgent 並不會直接要求 LLM 憑空猜測，而是啟動一套基於知識圖譜的參數尋找流程。該流程主要利用圖譜中定義的 REQUIRE_PARAMETER 與 RESPONSE_PARAMETER 關係，進行逆向推理，具體步驟如下：

1. **識別缺失參數：**系統首先辨識出當前 API 執行所需但數值未知的參數節點。
2. **追溯參數來源：**接著，系統於知識圖譜中逆向查詢，尋找哪些 API 的輸出 (RESPONSE_PARAMETER) 能夠提供該缺失參數。此過程相當於回答「哪些 API 的執行結果可作為此參數的值？」
3. **生成候選 API：**所有能提供該參數值的 API 均被視為候選 API。
4. **提交重新規劃：**這些候選 API 將連同參數需求，一同提交至重新規劃模組。LLM 會根據當前任務脈絡，選擇最合適的一個或多個 API，並將其整合進新的任務計畫中，生成專門用於獲取缺失參數的新步驟。

透過上述機制，APIKAgent 引導 LLM 必須在結構化知識庫中尋找答案，而非僅依賴其內部隱含知識。知識圖譜的存在，使參數的傳遞路徑清晰可追溯，確保 LLM 的決策過程「有據可循」，從而提升任務規劃的可靠性。

參數尋找範例

延續前面的範例，如圖3.3所示，假設在後續的任務規劃中，系統生成的計畫包含呼叫 `show_song_library`。在執行前的驗證階段，系統發現此 API 需要一個名為 `access_token` 的參數，但目前此參數的值是未知的。

- **識別缺失參數：**系統識別出 `show_song_library` API 需要 `access_token` 參數。
- **追溯參數來源：**系統查詢知識圖譜，尋找哪個 API 的 `RESPONSE_PARAMETER` 是 `access_token`。
- **生成候選 API：**查詢結果顯示，`spotify_login` API 的回傳結果中包含 `access_token`。因此，`spotify_login` 成為新的候選 API。
- **提交重新規劃：**系統將觸發重新規劃機制，並將「需要呼叫 `spotify_login` 以獲取 `access_token`」作為重新規劃的理由，同時將 `spotify_login` 加入候選 API 集合中。

此範例展示了知識圖譜如何協助系統在面對參數缺失時，並非隨機猜測，而是有條理地尋找解決方案，從而觸發更可靠的重新規劃流程。

3.3 APIKGAgent 的任務規劃與執行

本節旨在闡述本論文所提出的 APIKGAgent 如何將前一階段檢索到的候選 API，轉化為可執行的任務計畫。此流程不僅包含初始計畫的生成，更涵蓋了執行過程中的動態調整機制。首先，3.3.1 將說明系統如何生成初始任務計畫；接著，3.3.2 將詳述計畫的執行與錯誤監控；最後，3.3.3 將探討當執行遇到障礙時，系統如何動態地「重新規劃」，並透過「Pass 機制」優化執行效率。

3.3.1 任務規劃

任務規劃為 APIKGAgent 的決策核心，其目標在於利用大型語言模型 (LLM)，將候選 API 集合 C_{final} 與原始任務 T 自動轉譯為一結構化之任務計畫 P 。此計畫 P 由一個或多個步驟 s 組成，其生成方式如下：

$$P \leftarrow \text{LLM.GeneratePlan}(C_{final}, T)$$

$$P = \{s_1, s_2, \dots, s_n\}$$

為了處理 API 呼叫後的資料轉換與循環等複雜情境，任務計畫 P 中的每個步驟 s_i 不僅限於單純的 API 呼叫，亦可能包含資料處理、條件判斷等邏輯操作。此設計旨在確保任務能依預期執行，並靈活應對各種情境。

$$s \in \{s_j, s_p, s_{api}, s_{end}\}$$

其中 s_p 代表資料處理步驟，用於根據已有資料進行轉換或分析； s_{api} 代表 API 呼叫步驟，任何涉及使用外部 API 使用皆屬於此步驟； s_{end} 代表任務結束步驟，用於分析並回傳結果給使用者；而 s_j 則代表 3.1.2 節所闡述之「步驟跳躍機制」。

任務規劃範例

本節延續前述範例，假設 API 檢索階段已完成，候選 API 集合 C_{final} 包含 `show_song_library` 與 `show_song`。LLM 將基於此生成如後所述之任務計畫 P ：

任務計畫 P

- s_1 (API 呼叫): 呼叫 `show_song_library`，獲取使用者音樂庫中所有歌曲的 ID 列表。
- s_2 (API 呼叫): 根據 s_1 得到的所有歌曲 ID，呼叫 `show_song`，獲取所有歌曲的詳細資訊（包含按讚數）。
- s_3 (資料處理): 從 s_2 的結果中，提取每一首歌曲的按讚數 `like_count`，並且使用歌曲名稱與按讚數建立成一個列表。
- s_4 (資料處理): 比較 s_3 的列表，取得按讚數最高的歌曲。
- s_5 (結束步驟): 任務完成，返回 s_4 的歌曲名稱 `title`。

由於 LLM 生成的計畫偶爾會缺少必要資訊，所以為確保任務計畫的可執行性，本論文設計了檢查機制。系統會自動檢查每個步驟所需的資訊是否完整。若發現資訊缺失，系統將觸發參數尋找機制，從知識圖譜中探索能提供該資訊的 API。詳細檢查流程如 4.2.1.1 節所述。

3.3.2 任務計畫執行與錯誤監控

APIKGAgent 於生成任務計畫後，將依序執行計畫中的各個步驟 s_i 。本節將分別闡述各類步驟的執行方式與錯誤監控：

API 呼叫步驟

針對 API 呼叫步驟 s_{api} ，系統會依據計畫中定義的 API 名稱與參數，調用相應的 API。本論文在 API 步驟中包含兩種錯誤監控：

- **參數缺失監控**：系統在執行 API 前，會先透過 LLM 分析每個必填參數是否有它

的源頭，若沒有系統將觸發參數尋找機制，從知識圖譜中尋找能提供該參數的 API。若找到合適的 API，則將其納入 C_{final} ，並以此錯誤訊息重新生成任務計畫 P_{new} 。

- **API 呼叫結果監控：**系統將記錄並使用 LLM 分析執行結果，若 API 呼叫失敗，系統會分析錯誤原因；若 API 呼叫成功，系統會分析結果與步驟目標是否相同，若與預期不符，系統將尋找額外的 API 進行資訊擴增並重新規劃。

資料處理與跳躍步驟

針對資料處理步驟 s_p ，系統會根據前一步驟的執行結果，進行必要的資料轉換或計算。此步驟包含跳躍步驟 s_j ，其與一般資料處理步驟的差異在於處理結果，一般步驟會將處理結果紀錄在內部狀態中，而跳躍步驟則會根據條件決定是否跳轉至其他步驟。本論文提出的方法在資料處理步驟中包含兩種錯誤監控機制：

- **處理資訊充足性監控：**系統在進行處理資料前，會使用 LLM 分析目前資料是否足夠進行資料處理，若所需的資訊不完整，系統將記錄並分析缺失的資訊，並觸發 API 尋找機制以填補資訊缺口。
- **資料處理監控：**由於 LLM 帶有隨機性且有可能只輸出部分內容，系統會驗證資料處理後的內容是否符合預期，若不符合，則會重新處理資料直到成功，或直到重試次數達到預設的閾值。

結束步驟

對於到達結束步驟 s_{end} ，系統將終止任務執行流程，並將最終結果返回給使用者。

3.3.3 重新規劃與 Pass 機制

當任務執行過程遭遇參數缺失、API 呼叫失敗或其他異常時，APIKAgent 將自動觸發重新規劃機制。此機制旨在根據當前情境，動態生成一修正後之任務計畫。

此機制會將原始任務 T 、上次執行失敗的計畫 P_{old} 、失敗的步驟 s_{fail} 、失敗原因 r 、歷史所有失敗原因的集合 $\{r_i\}$ ，以及更新後的 API 集合 C_{final} ，一同提交給大型語

言模型，以生成新的任務計畫 P_{new} 。其過程可形式化為：

$$P_{new} \leftarrow \text{LLM.Replan}(T, P_{old}, s_{fail}, r, r_{i=1}^n, C_{final})$$

其中 P_{old} 為執行失敗的計畫， s_{fail} 為執行失敗或觸發重新規劃的步驟， r 為該次失敗的原因，而 $\{r_i\}$ 則記錄了所有歷史的失敗原因。

為提升執行效率，在新計畫 P_{new} 生成後，系統將啟用 3.1.3 節所闡述之「Pass 機制」。該機制會比對新舊計畫 (P_{new} 與 $P_{old,n}$) 的執行歷史。若新計畫中的某一步驟，其所呼叫的 API、語意目標及依賴的前置步驟皆與上一個舊計畫內某一已成功執行的歷史步驟等效，系統便將該步驟標記為「Pass」，並直接複用其歷史結果。此設計不僅避免了不必要的重複 API 呼叫與計算，亦防止了對外部狀態的意外重複修改。

重新規劃與 Pass 機制範例

本節將接續前述範例，探討兩種觸發重新規劃的情境：

情境一：因參數缺失而重新規劃

如 3.2.3 的範例所述，本論文提出的系統在執行計畫前發現缺少 `access_token`。參數尋找機制因而找到了 `spotify_login` API。

- **重新規劃理由 (r):** 需要先執行認證以獲取 `access_token`。`access_token` 可從 `spotify_login` API 的回傳結果中獲得。
- **更新後的 API 集合 (C_{final}):** `{show_song_library, show_song, spotify_login}`
- **生成的新計畫 (P_{new}):**
 - s'_1 (API 呼叫): 呼叫 `spotify_login`。
 - s'_2 (資料處理): 從 s'_1 的結果中提取 `access_token`。
 - s'_3 (API 呼叫): 呼叫 `show_song_library`，並傳入 s'_2 取得的 `access_token`。
 - ... (後續步驟與原計畫 P 的 s_2 至 s_5 相同)

在此情境下，因計畫尚未開始執行，舊計畫 P_{old} 無任何成功執行的步驟，故 Pass 機制無歷史結果可供複用。

情境二：因 API 執行失敗而重新規劃與 Pass 機制應用

假設新計畫 P_{new} 於執行過程中，步驟 s'_4 （即呼叫 show_song 以獲取歌曲資訊時）執行失敗。

- **重新規劃理由 (r):** show_song API 內部伺服器錯誤。
- **生成的新計畫 (P'_{new}):** LLM 可能生成與 P_{new} 完全相同的計畫，以進行重試。
- **Pass 機制介入：**
 - 系統比對新計畫 P'_{new} 與舊有計畫 P_{new} 的執行歷史。
 - 步驟 s'_1 、 s'_2 及 s'_3 均被識別為已成功執行且與新計畫中的對應步驟等效。
 - **執行跳過：**系統將此類步驟標記為「Pass」，並直接自記憶體載入其執行結果（如 access_token、歌曲列表）。
 - **恢復執行：**系統將從先前失敗的步驟（即 show_song API 呼叫）恢復執行。

此範例展示了 Pass 機制如何避免重新登入、重新獲取歌曲列表等耗時操作，從而提升錯誤恢復的效率。

第四章 APIKGAgent 系統實作

本章旨在詳述 APIKGAgent 的核心組件與運作流程，內容涵蓋知識圖譜構建、任務規劃、計畫執行、重新規劃，以及人為監督與介入機制。各節將深入探討系統的實作細節。

為有效管理 APIKGAgent 的複雜運作流程，本論文採用 LangGraph[21] 作為核心實作框架。LangGraph 是一個專為建構具備狀態（Stateful）的多步驟代理人（Agent）所設計的函式庫，其將代理人的運作流程模型化為一個狀態圖（State Graph），從而實現更清晰的邏輯控制與狀態管理。

4.1 知識圖譜構建

知識圖譜為 APIKGAgent 的知識核心，其以結構化方式整合 API 文件，為後續的任務規劃、API 檢索與自動化執行提供必要的語意支援。本節將闡述其設計目標、技術選型、資料結構，以及自動化建構流程。

本論文選用 Neo4j 圖形資料庫作為知識圖譜的儲存與查詢引擎，並結合 Python 進行自動化構建。此選擇主要考量圖形資料庫能直觀且高效地表達實體間的複雜關聯，並支援如 Cypher 等強大的圖形查詢語言。此外，在關鍵字抽取與語意分析環節，則整合大型語言模型（LLM）進行自然語言處理，以提升知識圖譜的語意豐富度與準確性。

知識圖譜的構建流程核心步驟如下：

1. **API 文件解析**：自動化解析 OpenAPI/Swagger 等標準格式的 API 文件，萃取 API 名稱、功能描述、請求參數、回傳格式等關鍵資訊，並進行標準化處理。
2. **關鍵字抽取與語意分析**：運用 LLM 對 API 描述進行深度語意分析，自動抽取具代表性的功能關鍵字，並建立對應的關鍵字節點。
3. **參數節點與依賴關係建立**：針對每個 API，分析其輸入與輸出參數，建立參數節點，並識別參數間的潛在依賴關係。
4. **節點與關係整合**：根據前述資訊，自動建立 API、關鍵字、參數等節點，並依據其語意關聯與依賴關係，在節點間建立連結。

5. **圖資料庫寫入：**將建構完成的知識圖譜轉換為 Cypher 陳述式，並寫入 Neo4j 圖形資料庫，以供後續的查詢與檢索。

詳細構建程式碼可參考附錄 B.1。

4.2 任務規劃模組

任務規劃為實現 APIKGAgent 自主性的關鍵環節，其核心目標是根據使用者賦予的自然語言任務，自動生成一系列有序、可執行的行動步驟。本節將闡述任務規劃的完整流程，以及最終產生的任務計畫結構。

4.2.1 任務規劃流程

任務規劃模組旨在將使用者輸入的自然語言任務，轉換為結構化的 API 調用計畫。此模組使用大型語言模型（LLM）進行語意理解與計畫生成。於規劃流程中，LLM 負責依據任務描述與從知識圖譜檢索出的候選 API 集合，產生初步計畫，並在必要時進行多輪迭代修正，以確保計畫的完整性與可行性。

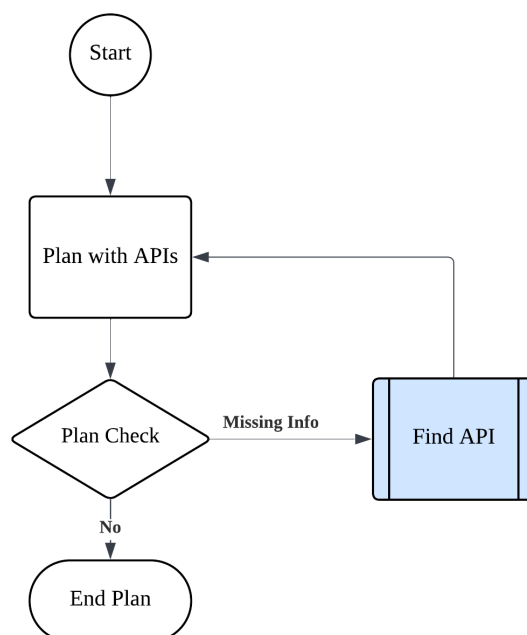


圖 4.1: 任務規劃流程圖

4.2.1.1 任務規劃流程

任務規劃的具體流程如圖 4.1 所示，其核心步驟如下：

1. **任務解析**：接收使用者的自然語言任務後，利用 LLM 進行語意解析，以準確理解任務的具體目標與隱含需求。
2. **候選 API 檢索**：根據解析後的任務需求，向知識圖譜發出查詢，檢索功能相關的候選 API 集合，作為後續規劃的基礎。
3. **初步計畫生成**：調用 LLM，根據任務目標與候選 API 列表，自動生成一組有序的 API 調用步驟。此計畫將明確各步驟的依賴關係與預期的輸入輸出。詳細規劃 prompt 如附錄A.1所示。
4. **計畫評估與迭代優化**：將初步計畫交由 LLM 進行可行性評估。若發現計畫存在邏輯缺陷或缺少必要資訊，將觸發迭代修正，此優化過程最多進行三輪，以確保計畫的品質。詳細計畫評估 prompt 如附錄A.2所示。
5. **結構化輸出**：最終計畫以標準化的 JSON 格式輸出，其內容包含步驟 ID、步驟類型、描述、依賴關係與輸出處理等詳細資訊，以供執行模組直接取用。

4.2.2 任務計畫的結構與內容

任務規劃階段的最終產出為一份結構化的任務計畫，此計畫是指導後續 API 調用與資料處理的核心藍圖。一份設計良好的任務計畫，應詳細記錄為達成使用者目標所制訂的完整行動策略，確保每一步驟都清晰、可執行。

一個標準的任務計畫步驟，通常包含以下幾個主要部分：

- **步驟 ID**: 於計畫內唯一標識一個行動步驟。
- **步驟類型**: 明確指出該步驟的性質，主要有以下：
 - EXECUTE_API: 表示此步驟的核心任務為進行 API 調用。

- **PROCESS_DATA**: 表示此步驟專注於對資料的內部處理，如格式轉換、邏輯運算或資訊萃取。跳躍步驟亦屬於此類型，主要用於根據條件決定是否跳轉至其他步驟。
- **STOP**: 表示此步驟為計畫的終止點，最後會進行結果的分析與輸出。
- **步驟描述**: 對該步驟的具體操作進行說明，以利後續執行模組理解其目的。
- **步驟依賴關係**: 指明執行該步驟所需的前置步驟。
- **輸出處理**: 定義如何處理該步驟的執行結果，例如：
 - **條件跳轉**: 根據執行結果，決定是否跳轉至計畫中的其他步驟，以實現循環或條件分支等複雜邏輯。
 - **結果儲存**: 預設情況下，每一步驟的執行結果都會被儲存至以 ID 作為標籤的步驟輸出，供後續步驟參考與使用。

透過此一結構化定義，任務計畫為後續的執行模組提供了清晰、無歧義的指令。最終產生的計畫將被傳遞給執行器，作為 API 呼叫與資料處理的直接依據。

4.3 任務計畫執行模組

本節將深入闡述 APIKGAgent 如何執行前一階段所生成的任務計畫。執行過程主要依據計畫中定義的步驟類型，分為「執行 API」、「資料處理」。

由於「終止步驟」僅分析結果並回傳最終答案，因此本節將重點介紹前兩類步驟的處理邏輯。終止步驟分析結果 prompt 如附錄 A.3 所示。

4.3.1 執行 API

在任務計畫中，API 的執行主要對應兩類操作：其一是透過 API 獲取外部資料；其二是執行 API 以改變遠端服務的內部狀態（如新增、刪除或更新資料）。本節將說明 APIKGAgent 處理這兩類 API 呼叫的通用機制。

APIKGAgent 執行 API 呼叫的詳細流程如圖 4.2 所示，其核心步驟如下：

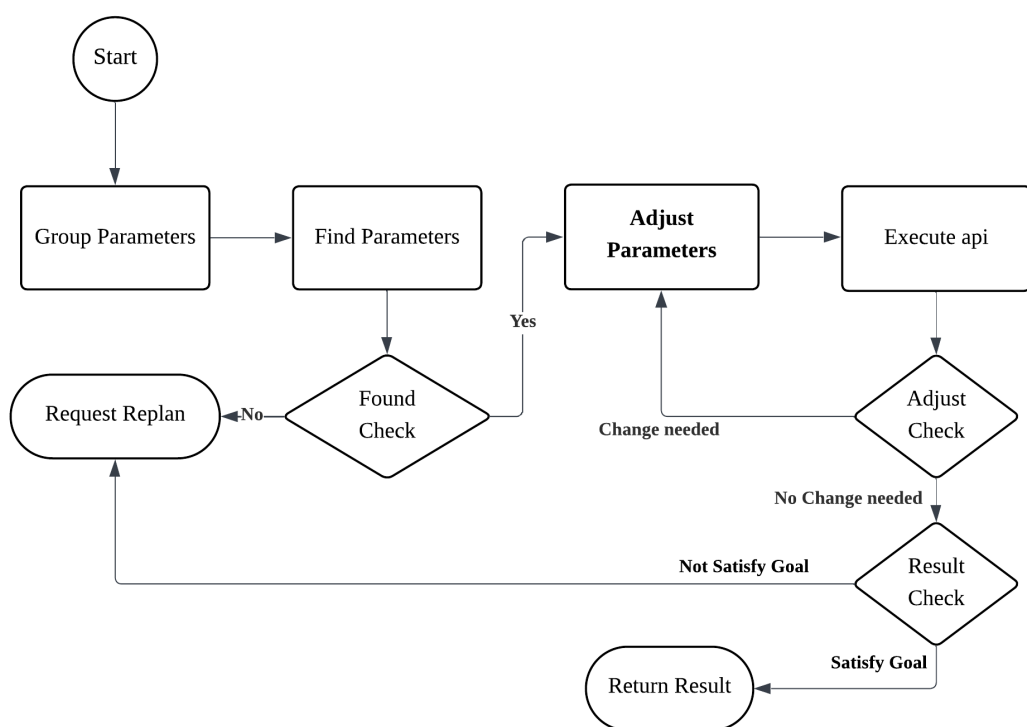


圖 4.2: API 執行流程圖

1. **API 內部參數相依性分析**：首先，辨識目標 API 內部參數之間的相依性，以便在後續步驟中能以正確的順序解析與填入參數。詳細參數相依性分析 prompt 如附錄A.4所示。
2. **必要參數尋找與填入**：從先前步驟的執行結果與記憶中，尋找當前 API 所需的必要（Required）參數。若必要參數未能尋獲，則判定當前計畫存在缺陷，觸發知識圖譜參數尋找並重新規劃。
3. **可選參數生成與處理**：在處理完必要參數後，進一步檢查 API 是否包含可選（Optional）的非必要參數。若存在，則運用 LLM 依據 API 文件及當前任務需求，動態生成合適的參數值。詳細可選參數生成 prompt 如附錄A.5所示。
4. **API 呼叫執行**：待所有參數準備就緒後，執行 API 呼叫，並獲取其回傳的原始結果。
5. **結果評估與參數迭代調整**：根據 API 的回傳結果，評估其是否滿足當前步驟的預期目標。若結果不符預期，且該 API 包含可調整的參數，系統將請求 LLM 調

整參數並返回步驟 3 重新執行，形成一個迭代優化的循環。詳細參數迭代調整 prompt 如附錄A.6所示。

6. **最終結果分析：**當 API 呼叫的結果滿足預期，或迭代調整功能認為已無需調整時，系統將對最終結果進行分析。若結果最終仍不滿足任務需求，則啟動重新規劃機制，以尋找替代的 API 或資料處理方法。詳細結果分析 prompt 如附錄A.7所示。

值得注意的是，在某些應用場景中，單一任務步驟可能需要對同一 API 進行多次呼叫（例如，遍歷一個列表並為每個項目調用 API）。為應對此類需求，APIKGAgent 在規劃階段即可將此類操作識別並設計為批次（Batch）模式，以簡化執行邏輯。在處理批次呼叫時，系統會特別考量其參數的動態相依性，確保每次呼叫均能從正確的來源獲取參數。

4.3.2 資料處理

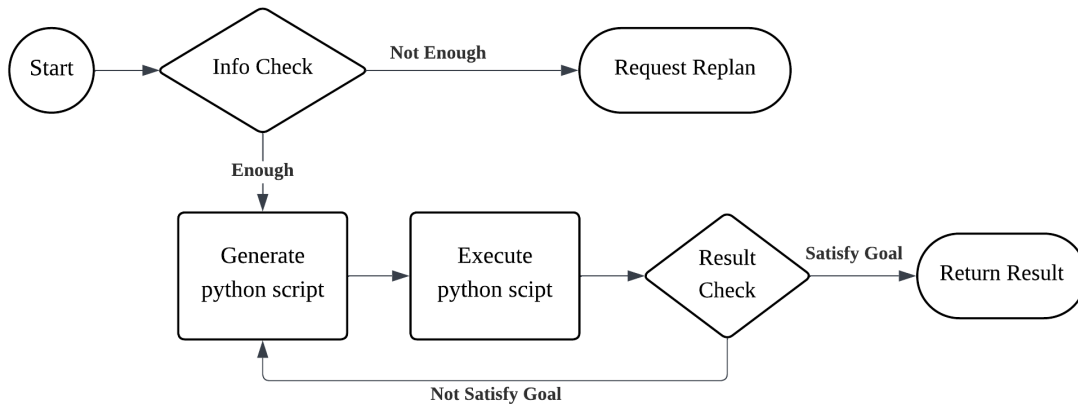


圖 4.3: 資料處理流程圖

資料處理為 APIKGAgent 任務計畫中的另一關鍵環節，專門負責對 API 回傳的資料或既有數據進行轉換、分析與處理，以達成特定的任務目標。此過程涉及輸入驗證、程式碼生成、沙盒執行與結果驗證等多個步驟。之所以採用 Python 進行資料處理而非直接使用 LLM，主要考量 LLM 在數學運算等邏輯處理上仍有其限制，尤其在處

理複雜數據結構或要求高精度計算時，直接使用 Python 程式碼能提供更高的可靠性。APIKGAgent 在資料處理階段的具體實作流程如圖 4.3 所示，其步驟說明如下：

1. **輸入資料完整性驗證**：在啟動資料處理前，首先檢核當前步驟所需的輸入資料是否齊備且格式有效。若偵測到資料缺失或不一致，將觸發重新規劃機制，以尋找能提供所需資料的 API 來補全資訊。詳細輸入資料完整性驗證 prompt 如附錄A.8所示。
2. **Python 程式碼動態生成**：確認輸入資料無誤後，根據當前步驟的具體任務需求，利用 LLM 動態生成相應的 Python 程式碼。此程式碼通常包含核心運算邏輯、資料結構轉換或特定的數據分析方法。詳細 Python 程式碼生成 prompt 如附錄A.9所示。
3. **沙盒執行與結果儲存**：生成的 Python 程式碼將在一個隔離且受控的沙盒（Sandbox）環境中執行，以確保系統安全。執行完成後，其產出結果將被結構化地儲存於對應步驟的記憶中，以供後續步驟使用。
4. **結果驗證與迭代修正**：為確保資料處理的準確性，系統會再次運用 LLM 對 Python 程式的執行結果進行驗證，判斷其是否符合預期格式並達成該步驟的目標。若驗證未通過，系統將啟動迭代修正循環，將失敗原因與上下文回饋給 LLM，並請求其重新生成或調整程式碼。此迭代過程將持續進行，直至處理結果符合要求，或達到預設的五次修正上限為止。詳細結果驗證 prompt 如附錄A.10所示。

4.4 重新規劃模組

為應對真實世界任務執行中普遍存在的不確定性，APIKGAgent 整合了一套重新規劃（Replanning）機制。此機制旨在當任務執行遭遇預期外事件、執行失敗，或因環境狀態變化導致原計畫不再適用時，能夠動態調整其行動策略。重新規劃的觸發與處理流程如圖 4.4 所示，涵蓋以下幾個階段：

1. **重新規劃上限確認**：代理人會先確認當前任務計畫是否已達到重新規劃的上限。若已達上限，則直接終止任務並回報失敗；若未達上限，則繼續進行下一步。

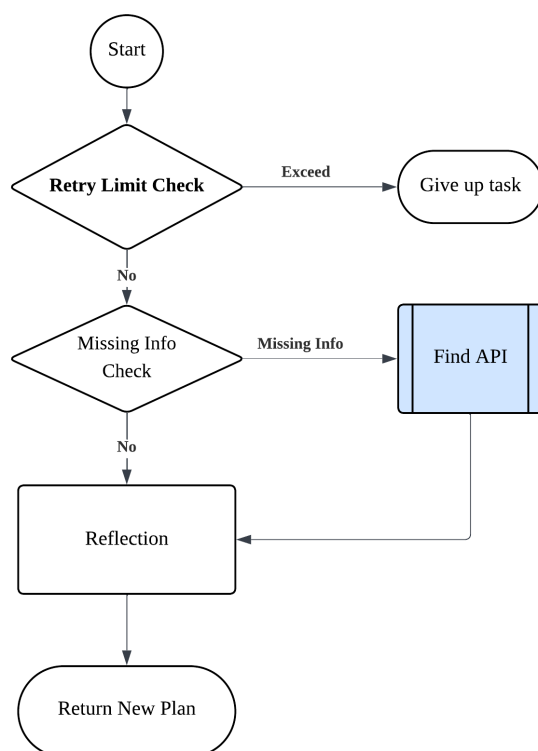


圖 4.4: 重新規劃流程圖

2. **資訊缺失分析**：若問題源於資訊缺失，代理人會將未滿足的資訊需求轉化為查詢條件，在知識圖譜中搜尋能提供該資訊的 API。
3. **修正計畫生成**：此階段利用 LLM 作為核心規劃引擎，生成修正後的任務計畫。LLM 的輸入涵蓋了原始任務目標、從知識圖譜檢索的候選 API 集合、先前的失敗計畫、已識別的執行問題，以及已成功執行的步驟與其結果。透過豐富的上下文，引導 LLM 生成一個能解決當前困境，並能有效引導任務朝向最終目標的新行動序列。

透過重新規劃機制，APIKGAgent 得以在面對執行挑戰與環境動態變化時，展現出更高的靈活性與適應性，從而提升任務的最終成功率。詳細重新規劃 prompt 如附錄A.11所示。

4.5 人為監督與介入機制

儘管 APIKGAgent 的設計目標是追求高度的自主性，但在複雜或高風險的真實應用場景中，完全自動化可能面臨潛在的挑戰與風險。為確保任務執行的安全性、可靠性並使其最終結果符合使用者意圖，本論文設計了一套人為監督與介入（Human-in-the-loop）機制，允許使用者在關鍵決策節點對代理人的行為進行審視、引導與修正。

本論文提供的人為介入機制主要體現在以下兩個關鍵階段：

1. **任務規劃審查與調整**：在代理人完成初步規劃或重新規劃後，系統會將生成的任務計畫呈現給使用者進行審查。此階段的介入賦予使用者以下權力：

- **計畫合理性評估**：使用者可憑藉其領域知識與經驗，評估計畫的邏輯連貫性、步驟的必要性與整體可行性。
- **API 選擇審查**：使用者能檢視計畫中選用的 API 是否符合預期，以避免因 LLM 對 API 功能的錯誤理解而導致執行偏差。
- **計畫優化與手動調整**：使用者可以直接修改計畫，例如調整步驟順序、替換 API、增刪步驟，或引入外部知識以指導代理人生成更優的計畫。

此階段的介入，不僅能提升計畫的品質，也確保了代理人的後續行動與使用者真實需求的高度一致性。

2. **高風險操作確認**：對於可能產生顯著影響或不可逆後果的操作（例如，資料刪除、服務配置修改等），系統將其定義為高風險操作。在執行此類 API 前，系統將暫停執行流程，並主動請求使用者進行確認。

- **警示與資訊呈現**：系統會標示出即將執行的高風險 API 及其關鍵參數，以提供使用者檢查。
- **執行決策權移交**：使用者擁有最終的決策權，可選擇允許執行、拒絕執行同時告知理由。此設計確保了關鍵決策由人類智慧做出，有效規避了全自動化系統可能帶來的潛在風險。

這種介入策略對於保障系統安全、防止意外發生，以及維持使用者對系統的最終控制權至關重要。

總結而言，人為監督與介入機制的設計，旨在於自動化效率與人工智慧的可靠性、可控性之間尋求最佳平衡。它不僅提升了 APIKGAgent 在複雜或模糊情境下的適應能力，也為處理全自動化系統難以獨立解決的問題提供了必要的應對手段。因此，此機制是確保本論文所提出的方法能夠在真實世界中安全、有效部署的關鍵設計之一。

4.6 其他實作細節

本節將闡述 APIKGAgent 實作中其他重要的輔助性設計，包含引導大型語言模型 (LLM) 的提示詞模板，以及確保系統狀態持久性的記憶管理機制。

4.6.1 提示詞模板

本節旨在說明 APIKGAgent 中引導大型語言模型 (LLM) 生成預期輸出的提示詞模板 (Prompt Template)。此模板為與 LLM 互動的核心，透過結構化的設計，確保模型能準確理解任務脈絡與限制，從而產生一致且有效的回應。所有提示詞均遵循此模板進行設計，並根據特定任務需求進行客製化。

提示詞模板

```
{system_message}
# Instructions
此區域提供 LLM 執行任務或功能的明確指示，並包含相關限制與注意事項。

# Steps
此區域採用思維鏈（Chain of Thought, CoT）方法，將任務拆解為多個清晰的
↳ 步驟，引導 LLM 進行推理。

# Output format
此區域定義 LLM 的輸出格式與結構，通常採用 JSON 格式以確保輸出的一致性
↳ 與可解析性。

# Examples
此區域採用少樣本學習（Few-shot Learning）方法，提供具體範例以協助 LLM
↳ 理解任務的執行方式。

# Notes
此區域補充說明執行任務時的特殊要求或限制，確保 LLM 能遵循特定規則生成
↳ 輸出。

{user_message}
此區域提供 LLM 處理的具體輸入或問題，其內容依實際任務而定。
```

4.6.2 記憶管理

為確保 APIKGAgent 運作的連續性與穩定性，本論文設計了一套記憶管理機制。此機制將記憶分為兩類：

- **使用者資訊：**用以儲存如使用者提供的地址、聯絡方式等長期性資料。
- **步驟結果：**用以記錄各執行步驟的輸出結果，此類資訊僅在後續步驟明確請求時提供給 LLM。

所有記憶皆以 LangGraph 的狀態（State）物件格式儲存，並存於 SQLite 中。此設計不僅能在系統意外中斷後恢復執行狀態，保障了運作的穩定性，同時也確保了使用者資料的持久性。

第五章 實驗與結果分析

5.1 研究問題

為系統性地驗證 APIKGAgent 的設計理念與實際效能，本論文將圍繞以下四個核心問題（Research Questions, RQs）展開：

- **RQ1：APIKGAgent 的整體任務表現：**評估本論文提出的 APIKGAgent 框架，在面對複雜任務時的綜合表現與完成能力。
- **RQ2：知識圖譜是否能成功篩選出任務相關 API：**探討以知識圖譜作為 API 檢索基礎，是否能為任務篩選出正確的 API 集合。
- **RQ3：錯誤監控機制的影響：**驗證透過大型語言模型（LLM）監控另一 LLM 的輸出內容以及 API 的執行結果，是否能實質提升整體的任務完成率。
- **RQ4：思維鏈的影響：**評估 APIKGAgent 有無引入思維鏈（Chain of Thought, CoT）機制，是否對於大型語言模型輸出有所提升。

5.2 實驗設置

5.2.1 實驗基準

本論文採用 AppWorld [20] 作為核心的實驗基準與測試平台。該平台模擬了真實世界的應用程式互動情境，內建 9 項服務與 457 支 API。AppWorld 提供了多樣化的測試任務，其設計旨在評估代理人執行需整合多支 API 方能完成的複雜指令之能力，這使其成為驗證 APIKGAgent 效能的理想環境。

5.2.2 評估指標

本實驗的核心評估指標為**任務完成率 (Task Goal Completion, TGC)**。一個任務被判定為「成功」，必須滿足以下所有條件：

1. **目標達成**：代理人最終的產出符合任務指令的預期目標。
2. **狀態正確**：根據 AppWorld 平台的最終資料庫狀態進行比對，確認所有相關資料均已正確修改或查詢。
3. **無副作用**：代理人在執行過程中，未對任務目標以外的資料庫內容造成任何非預期的更動。

此一綜合性指標能客觀反映 APIKGAgent 在準確理解任務、正確調用 API 及確保操作安全性方面的綜合能力。

另一指標為**情境完成率 (Scenario Goal Completion, SGC)**，在資料集中，一個任務情境包含多個相似類型的任務。若代理人成功完成所有相關任務，則該情境被視為成功。此指標有助於評估代理人在處理相似任務中的穩定度。

5.2.3 實驗環境

為確保實驗的一致性與可比較性，所有實驗均在統一的環境下執行，詳細配置如表5.1與表5.2：

表 5.1: 實驗硬體與軟體環境

項目	設定值
作業系統	Ubuntu 24.04
CPU	Intel Core i7-11700
記憶體	40 GB RAM
程式語言	Python 3.12

表 5.2: 代理人設定

項目	設定值
使用 LLM	gemini-2.5-flash-preview-04-17 (規劃/判斷模型)
最大重新規劃次數	gemini-2.0-flash (評分)
Temperature 參數	10 次
Thinking Budget	規劃與判斷模型: 0.7
	評分模型: 0.1
	2048 tokens

5.3 實驗一

本節旨在回應 RQ1，評估 APIKGAgent 於複雜任務中的綜合表現。為此，本實驗採用 AppWorld 測試平台中的 test_normal 資料集進行實驗。該資料集包含 168 個任務，模擬真實世界中需要多步驟、多應用程式協作的複雜情境。

為衡量 APIKGAgent 的效果，本論文將其任務完成率與多個現有方法進行比較，包含 ReAct [1]、PlanExec [11]、FullCodeRefl(生成完整程式碼再透過 Reflexion[22] 調整)、以及 Iterative Parallel Function Calling (IPFunCall) [23]，其實驗數據引用自 AppWorld [20] 的研究成果。此外，本論文亦納入 LOOP [17] 方法作為比較基準，其數據則源於其原始論文。詳細的比較結果如表 5.3 所示。

表 5.3: 各方法於 AppWorld test_normal 資料集的表現

LLM	方法	TGC (%)	SGC (%)
GPT-4o	ReAct	48.8	32.1
	PlanExec	44.6	23.2
	FullCodeRefl	33.9	26.8
	IPFunCall	32.1	16.1
llama3-70B	ReAct	20.8	8.9
	PlanExec	8.9	1.8
	FullCodeRefl	24.4	17.9
Qwen 2.5 32B	ReAct	39.2	18.6
	LOOP	72.6	53.6
Gemini 2.5 flash	ReAct	50.6	28.6
	FullCodeRefl	36.9	26.8
	APIKGAgent	41.7	17.9

根據表 5.3 的實驗結果，APIKGAgent 在 test_normal 資料集上達成了 41.7% 的任務完成率 (TGC) 與 17.9% 的情境完成率 (SGC)。為深入探討其潛在的改進空間，本論文對其執行過程中的失敗案例進行了分析，發現問題主要可歸結為以下幾種類型：

1. **大型語言模型未遵循指令格式**：本系統要求大型語言模型 (LLM) 將計畫輸出為特定的結構化格式 (如 4.2.2 所述)，然而 LLM 在生成回應時偶爾會忽略或變更格式，導致系統無法正確解析計畫，進而影響後續執行。未來可嘗試將附加格式需求用額外步驟加入，而不是直接在提示詞中要求 LLM 遵循，這樣可以減少 LLM 在格式上的偏差。

2. **錯誤處理機制不完善與過早放棄：**在某些情況下，規劃模型在遇到初步失敗後，未嘗試修正或尋找缺失資訊，而是直接放棄，顯示錯誤處理機制仍有改進空間。未來可以針對不同錯誤產生額外的子目標或是重試此步驟，引導代理人進行更深入的錯誤修正。
3. **規劃與理解的偏差：**代理人的規劃模型（Planner）偶爾會遺漏或誤解任務指令中的細微約束。例如，在一個要求「發送訊息給尚未註冊 Venmo 的家庭成員」的任務中，代理人雖能正確發送訊息，卻未能篩選出未註冊的成員，導致任務目標未完全達成。另一種情況是，當任務指令本身存在模糊性時，代理人的解讀可能與預期結果產生偏差。
4. **執行流程的瑕疵：**在執行方面，代理人有時會因步驟敘述不夠清晰或缺乏前序步驟的上下文資訊而受到影響。例如，在一個要求從訊息中擷取特定資訊的任務中，若步驟僅籠統地描述為「擷取訊息中的資訊」，卻未指明具體內容，可能導致代理人僅擷取部分資訊，從而無法完成後續步驟。
5. **API 的誤用：**LLM 對 API 的理解存在偏差，同樣是導致任務失敗的原因之一。例如，在一個任務中，`place_order` API 會將購物車內所有商品統一結帳，但 LLM 在規劃時未考慮此特性，導致非預期的商品被一併購買，使任務失敗。
6. **規劃步驟的邏輯謬誤：**在部分案例中，代理人生成的計畫步驟在邏輯上存在矛盾或不合理之處。例如，在一個要求「持續播放下一首歌曲，直到播放的歌曲是使用者曾按讚的歌曲為止」的任務中，代理人可能生成一個包含無效跳轉指令的步驟，如：「步驟 5：檢查前一步驟的歌曲是否在按讚清單中，若否，則跳轉至步驟 3」。然而，若步驟 3 與 4 並非「播放下一首歌曲」的相關操作，此一設計將導致代理人陷入無效的邏輯迴圈，無法完成任務。
7. **複雜流程的規劃挑戰：**在處理某些涉及多重條件的複雜任務時，代理人難以在初始規劃階段預見所有執行細節。例如，在執行付款時可能先遭遇餘額不足，儲值時又發現支付卡過期。當代理人根據初步執行結果調整規劃時，可能衍生出新的問題。若重新規劃次數過多，LLM 可能會遺忘部分限制條件或任務細節，導致相同的錯誤在後續執行中反覆出現。未來一樣可考慮針對錯誤設定子目標去修正，

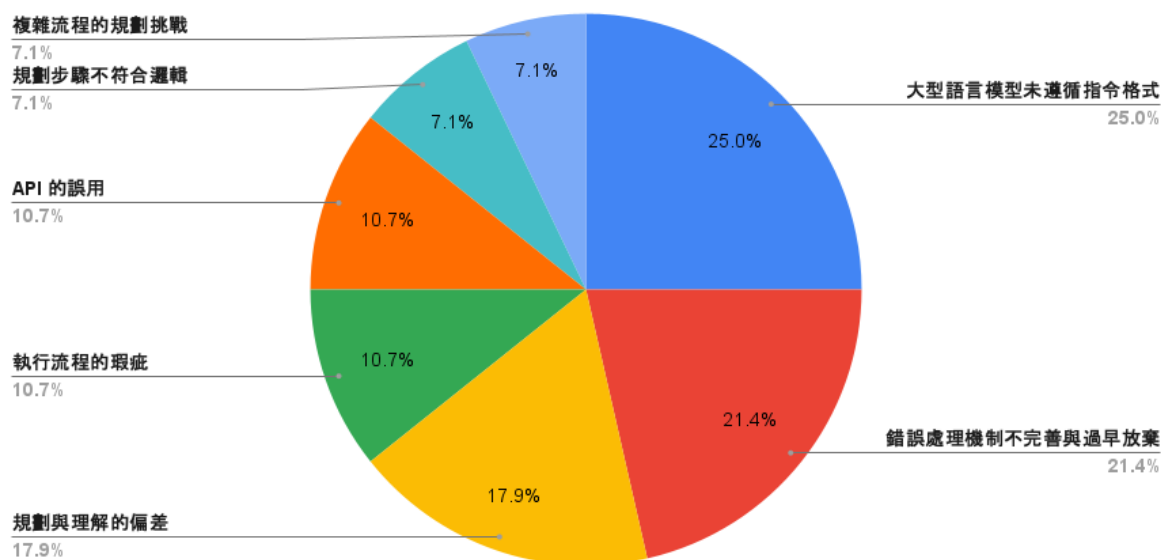


圖 5.1: APIKGAgent 在 dev 資料集中的錯誤類型分佈

使錯誤處理與任務執行分離，避免 LLM 在多次修正後遺忘初期的任務細節。

為進一步釐清各類錯誤的影響程度，本論文針對 dev 資料集中的 57 個任務進行了錯誤類型統計，如圖 5.1 所示。數據顯示，佔比最高的錯誤類型為「大型語言模型未遵循指令格式」(25.0%)，其次是「錯誤處理機制不完善與過早放棄」(21.4%) 與「規劃與理解的偏差」(17.9%)。此結果表明，儘管 APIKGAgent 在多數情況下能正確理解任務並調用 API，但在規劃的細緻度、格式遵循以及錯誤修正等方面，仍存在顯著的改進空間。

此外，本論文亦分析了代理人的重新規劃行為。如圖 5.2 所示，大部分任務只需兩次內的修正即可完成，而成功的任務中最大重新規劃次數為 9。然而，有 18 個任務在達到 10 次重新規劃上限後仍未成功。手動分析顯示，這些失敗案例多數源於 LLM 在多次修正後，遺忘了初期的任務細節或限制條件，陷入「修復 A 問題卻引發 B 問題，再修復 B 問題卻又導致 A 問題復現」的循環。此現象說明，單純增加重新規劃次數並非解決此類複雜任務失敗的有效途徑。

綜合上述分析，針對研究問題一 (RQ1)，本論文的 APIKGAgent 在 AppWorld 的 test_normal 資料集上取得了 41.7% 的任務完成率。雖然此數值未超越所有基準方法，但其模組化的架構設計揭示了獨特的價值。該架構使其具備監控與獨立調整各環節的能力，在面對高度複雜的任務時，能針對特定瓶頸進行優化，賦予了系統更高的擴充

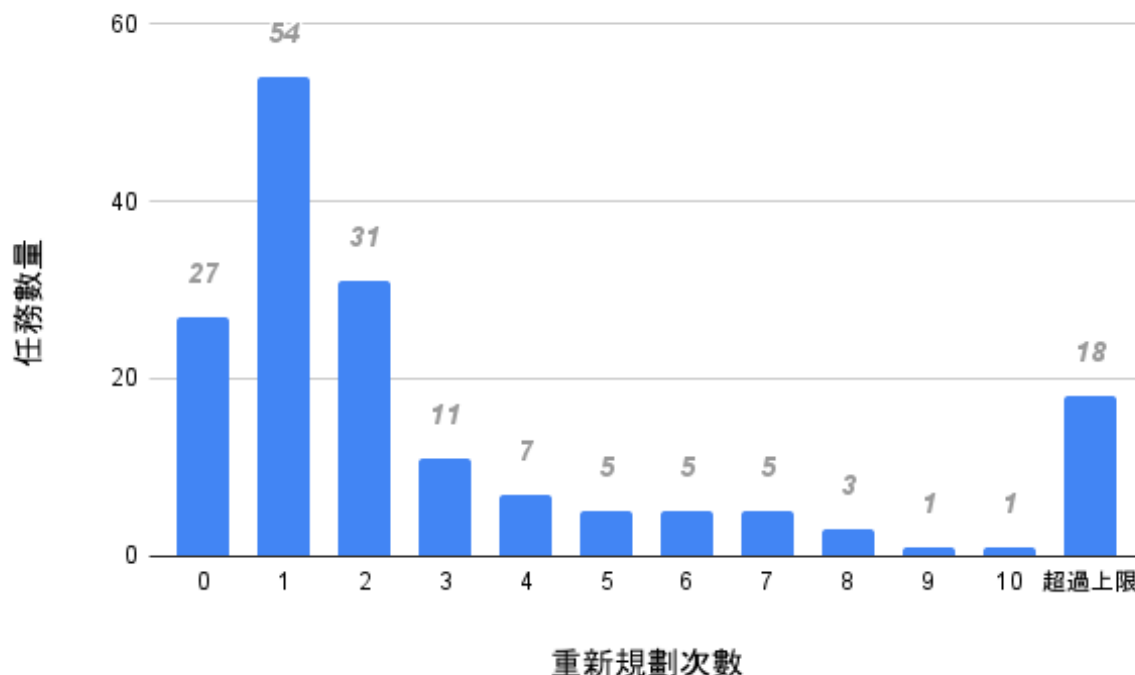


圖 5.2: APIKAgent 在 test_normal 資料集上各任務的重新規劃次數

性與應用潛力。此外，其「先規劃、後執行」的策略，不僅提升了執行過程的可控性，也讓使用者能更容易地理解代理人的行為模式，並在必要時進行介入調整。

Token 數與執行時間分析

為深入評估 APIKAgent 的運作成本與效率，本節將分析其在執行 test_normal 資料集任務時的 Token 使用量與執行時間。

如圖 5.3 所示，APIKAgent 的平均總 Token 使用量為 388,378，其中輸入 Token 佔 367,007，輸出 Token 則為 21,371。輸入 Token 佔比較高的原因主要有兩個：其一是在 API 評分階段，評分系統的每次 LLM 呼叫均包含了完整的任務指示、思維鏈（CoT）及少樣本學習（Few-shot Learning）範例，導致越多個 API 要進行評分時，雖然單次呼叫的輸入 Token 數變少，但是總 Token 數增加；其二是在執行階段為進行參數選擇，代理人會將完整的 API 呼叫結果納入上下文，以供 LLM 進行分析。為優化這兩個問題，未來可研究批次評分機制，將多個 API 呼叫結果合併後再一次性評分。同時，在參數選擇階段，可考慮讓模型優先讀取 API 結果的摘要或 Schema，僅在必要時才查閱完整內容，以有效降低輸入 Token 的耗用。

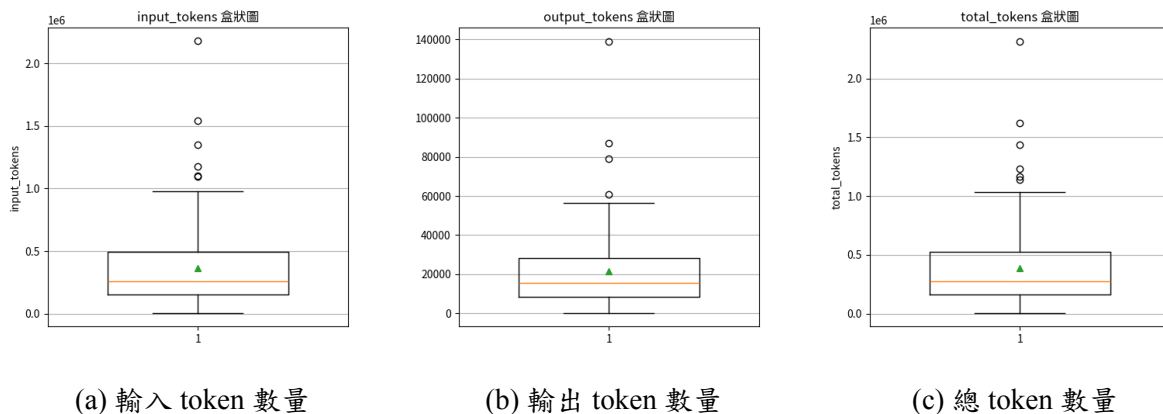


圖 5.3: APIKGAgent 在 `test_normal` 資料集上的使用 token 數量分佈 (a: 輸入, b: 輸出, c: 總數), 綠色三角形表示平均值。

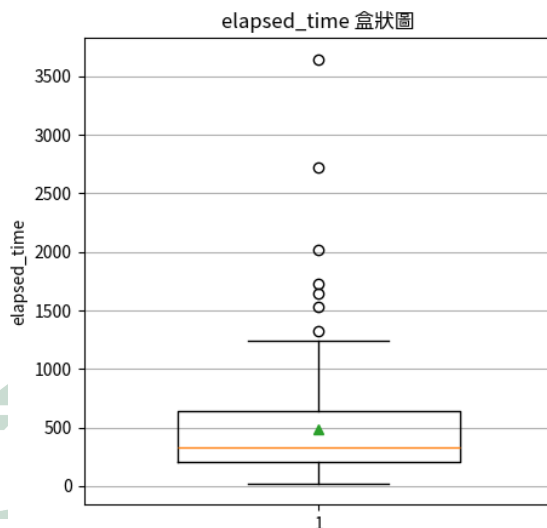


圖 5.4: APIKGAgent 在 `test_normal` 資料集上執行時間數量分佈, 綠色三角形表示平均值。

在執行時間方面, 如圖 5.4 所示, APIKGAgent 的平均任務完成時間為 483 秒。此時間成本主要源於其循序式的運作流程, 即嚴格遵循「規劃、執行、重新規劃」的循環。為提升執行效率, 未來可引入並行處理機制, 例如參考 LLMCompiler [24] 的方法, 識別並同時執行計畫中無相互依賴關係的步驟, 從而縮短整體等待時間。

5.4 實驗二

為探討 APIKGAgent 中各項關鍵設計的實際效益, 本節將進行一系列的消融實驗, 以分別回答 RQ2 與 RQ3。本實驗旨在透過移除特定模組, 來量化評估其對整體任務完

成率的影響。本實驗採用 AppWorld 的 test_normal 資料集中 56 個不同情境的任務。最終實驗結果如表 5.4 所示。

表 5.4: 各組件消融對任務完成率之影響

消融設定	TGC (%)
完整架構	46.4
無知識圖譜	37.5
無錯誤監控機制	42.9

知識圖譜的效益分析 (RQ2)

為驗證知識圖譜於 API 檢索與參數生成的效益，本論文設計了「移除知識圖譜」的實驗組。在此設定下，API 的選擇流程轉為由 LLM 直接分析任務需求，並提供相關服務中的所有 API 給規劃器進行使用。同時，API 呼叫的參數生成亦改為由 LLM 根據任務目標與記憶內容直接產生，而不會逐一選擇與調整參數。

實驗結果顯示，移除知識圖譜後，任務完成率由 46.4% 下降至 37.5%。經過手動分析發現，錯誤率的增加主要源於 LLM 在一次性生成所有參數時，可能會忽略部分必要參數。而在有知識圖譜的情況下，任務完成率並沒有下降，這表明知識圖譜能有效地篩選出與任務相關的 API，相對於直接給予規劃器所有 API，只傳遞與任務相關的 API 能減少 LLM 在進行規劃時的輸入上下文長度。

錯誤監控機制的效益分析 (RQ3)

為評估錯誤監控機制的實質貢獻，本實驗移除了 APIKGAgent 中的三項核心檢查點：(1) 規劃後的缺失資訊檢查；(2) 資料處理前資訊充足性監控；(3) API 呼叫結果監控。

實驗結果表明，在缺乏錯誤監控機制的情況下，任務完成率從 46.4% 下降至 42.9%。此發現回應了 RQ3，證明錯誤監控機制對於提升任務完成率具有正面助益。該機制能有效地在執行過程中提早發現錯誤、總結原因、驗證資料適用性，並對 API 結果進行分析，從而引導代理人進行即時修正，確保任務朝正確方向推進。

5.5 實驗三

本節旨在回應 RQ4，深入探討在 APIKGAgent 中，2.1.4 小節所提的提示詞 (prompt) 中的思維鏈 (Chain of Thought, CoT) 對於大型語言模型 (LLM) 在任務規劃與判斷能力上的實質影響。由於本論文的模型採用 Gemini 2.5 flash 作為規劃與判斷模型，該模型可進行開關思考模式機制，因此本實驗會將會有四組實驗設定，分別為：

- **完整架構 (CoT + 思考模式)**：採用包含明確 CoT 引導的提示詞，並開啟模型的內建思考模式。
- **無 CoT + 有思考模式**：移除提示詞中的 CoT 引導，但保留模型的內建思考模式。
- **有 CoT + 無思考模式**：保留提示詞中的 CoT 引導，但關閉模型的內建思考模式。
- **無 CoT + 無思考模式**：同時移除提示詞中的 CoT 引導，並關閉模型的內建思考模式。

本實驗同樣採用 AppWorld 的 test_normal 資料集中 56 個不同情境的任務進行測試。

各設定下的實驗結果如表 5.5 所示。

表 5.5: 思維鏈與思考模式對任務完成率的影響

實驗設定	TGC (%)
完整架構 (CoT + 思考模式)	46.4
無 CoT + 有思考模式	37.5
有 CoT + 無思考模式	23.2
無 CoT + 無思考模式	12.5

數據顯示，完整架構的任務完成率為 46.4%。當移除提示詞中的 CoT 設計後，完成率下降至 37.5%；若僅保留 CoT 而關閉模型思考模式，完成率則降至 23.2%；而兩者皆無的情況下，完成率僅有 12.5%。此結果清晰地表明，CoT 設計與模型內建的思考模式對任務成功率均有顯著的正面影響，且模型內建思考模式的貢獻尤為突出。

為進一步探究任務失敗的根本原因，本論文對各消融設定下的失敗案例進行了分析，歸納出兩類核心問題：

- **移除 CoT 導致指令遵循能力下降：**在缺乏明確 CoT 引導時，LLM 更容易產出不符合預期格式的計畫，或是在規劃時忽略任務的特定限制條件，導致計畫內容過於籠統或出現邏輯錯誤。
- **關閉思考模式導致判斷能力減弱：**在關閉模型內建的思考模式後，LLM 對於當前資訊狀態的判斷能力顯著下降。例如，模型可能無法辨識一個檔案內容是已知的歌曲清單，而錯誤地判斷為資訊不足，從而中斷了原本應繼續的處理流程。

這兩類錯誤的影響，可以從圖 5.5 中各設定達到重新規劃次數上限的任務數量得到印證。判斷能力下降會使代理人陷入「重複判斷資訊不足」的循環；而指令遵循能力下降則會導致代理人「反覆生成錯誤計畫」。兩者均會快速消耗重新規劃的次數，最終導致任務失敗。

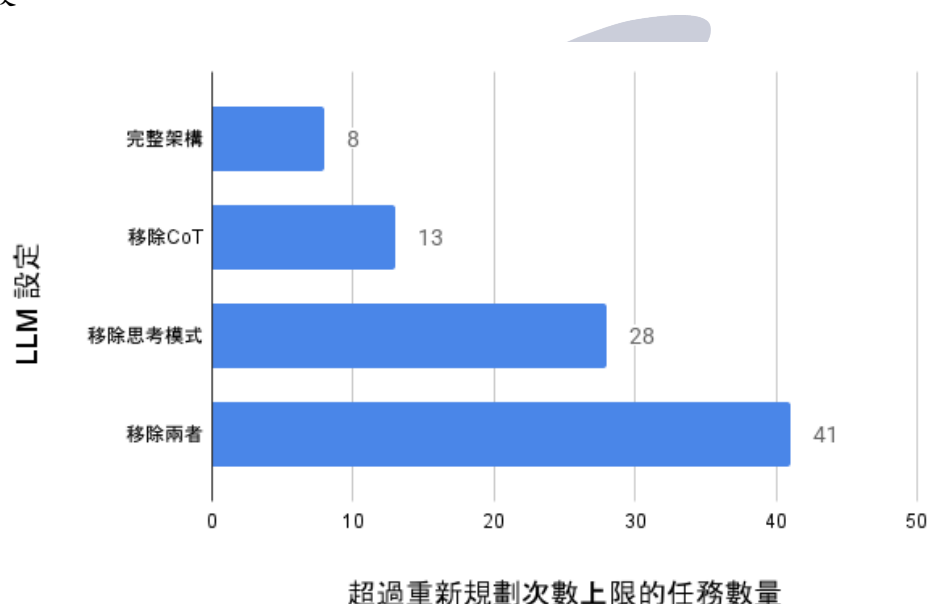


圖 5.5: 不同實驗設定下，因達到重新規劃次數上限而失敗的任務數量

綜合而言，本實驗結果回應了研究問題四（RQ4），證實了思維鏈（CoT）機制的有效性。儘管如 Gemini 2.5 flash 等先進模型已具備強大的內建思考能力，但在提示詞中提供明確的 CoT 結構，依然能提升其指令遵循的穩定性與規劃的細緻度。

第六章 結論與未來研究方向

6.1 結論

本論文旨在應對現實中複雜的數位工作流程自動化挑戰，提出一個名為 APIKGAgent 的自主代理人框架。此框架整合大型語言模型、知識圖譜與結構化工作流程管理，使其能自主地解析使用者以自然語言描述之任務，並透過多個 API 的協同合作以完成目標。

為實現此一目標，本論文的核心貢獻在於提出以知識圖譜為基礎的 API 相依性模型。我們不僅將非結構化的 API 文件轉換為機器可讀的結構化知識，更重要的是，我們藉此建構了 API 之間的複雜依賴關係。此模型為代理人提供了超越單純 API 檢索的深度理解，使其能更精確地生成執行計畫。在此基礎上，我們設計了「規劃、執行、監控、修正」的工作流程，透過先規劃後執行的策略，提升了任務執行的可控性與透明度；並藉由錯誤監控與重新規劃機制，賦予代理人應對執行錯誤與環境變化的動態適應能力。

實驗結果初步驗證了本論文所提方法的潛力。在 AppWorld 的基準測試中，本論文提出的方法雖未在所有指標上超越現有方法，但其設計旨在解決更根本的 API 協作挑戰。更重要的是，透過消融實驗，本論文量化地證明了以知識圖譜為基礎的依賴關係模型與參數尋找模式，對於提升整體任務成功率具有正面效益。

總體而言，本論文成功地建構了一個功能完整的自主代理人框架，並透過系統性的實驗，驗證了其設計理念與實際效能。APIKGAgent 不僅為自動化 API 協作任務提供了一套可行的解決方案，其模組化的架構也為未來的學術研究與實際應用，奠定了良好的擴充基礎。

6.2 未來研究方向

本論文雖然在基於大型語言模型的代理人（Agent）自動化工作流程上獲得初步成果，然仍有多個層面可供未來研究持續深化與探索。

知識圖譜優化

目前系統中的知識圖譜在 API 搜尋與依賴關係表達上仍有精進空間。未來研究可朝向兩個方向進行：其一，強化知識圖譜的語意搜尋能力，讓搜尋不僅限於關鍵字匹配，更能理解複雜的查詢意圖，以提升 API 檢索的準確性。其二，應用不同搜索方法，如 AutoKG [25]，使搜索的速度更為高效，使得代理人能在更短時間內獲得所需的 API 資訊，從而加快整體工作流程的執行速度。

規劃器功能增強

為提升代理人解決複雜任務的能力，規劃器（Planner）的功能可進一步增強。首先，可以將遇到的錯誤與改進的方向自動拆解為一系列具體的子目標，使代理人能先完成小目標，避免因目標過於龐大而陷入困境。其次，應建立更嚴謹的驗證機制，在規劃階段即對每一步驟所需的參數、格式與前置條件進行檢查，確保計畫的可行性與正確性，從而提升工作流程的穩定性。

多輪對話與任務銜接

現實世界中的任務往往具有連續性。因此，為代理人擴充多輪對話（Multi-turn Conversation）的能力至關重要。未來的研究應著重於如何讓代理人理解對話的上下文，並將已完成任務的成果或狀態，無縫地銜接至下一個新任務中，實現更流暢、更具智慧的人機協作。

錯誤處理與系統穩定性

隨著代理人自主性的提高，錯誤處理機制也變得格外重要。若代理人因錯誤地使用 API 而導致非預期的負面影響，系統應具備相應的應對措施。未來的研究可以探索如交易復原（Transaction Rollback）機制的引入，或在沙盒（Sandbox）環境中執行高風險操作，以確保系統在面對錯誤時的強健性與安全性。

效能問題的改善

本論文所提出的方法在執行效率上仍有進步空間，尤其步驟較長時，其循序執行的特性可能導致較長的運行時間。為有效改善此一效能瓶頸，未來研究可考慮引入 LLMCompiler [24] 的優化策略。LLMCompiler 的核心思想是透過分析模型執行步驟，將沒有互相依賴的步驟進行平行處理，從而顯著提升運作速度。



參考文獻

- [1] Shunyu Yao et al. “ReAct: Synergizing Reasoning and Acting in Language Models”. In: *The Eleventh International Conference on Learning Representations*. 2023. URL: https://openreview.net/forum?id=WE_vluYUL-X.
- [2] Hugo Touvron et al. “Llama 2: Open Foundation and Fine-Tuned Chat Models”. 2023. arXiv: 2307.09288 [cs.CL]. URL: <https://arxiv.org/abs/2307.09288>.
- [3] Ashish Vaswani et al. “Attention Is All You Need”. 2023. arXiv: 1706.03762 [cs.CL]. URL: <https://arxiv.org/abs/1706.03762>.
- [4] Yujia Qin et al. “ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs”. 2023. arXiv: 2307.16789 [cs.AI]. URL: <https://arxiv.org/abs/2307.16789>.
- [5] Shishir G. Patil et al. “Gorilla: Large Language Model Connected with Massive APIs”. 2023. arXiv: 2305.15334 [cs.CL]. URL: <https://arxiv.org/abs/2305.15334>.
- [6] Qingxiu Dong et al. “A Survey on In-context Learning”. 2024. arXiv: 2301.00234 [cs.CL]. URL: <https://arxiv.org/abs/2301.00234>.
- [7] Jason Wei et al. “Chain-of-Thought Prompting Elicits Reasoning in Large Language Models”. 2023. arXiv: 2201.11903 [cs.CL]. URL: <https://arxiv.org/abs/2201.11903>.
- [8] Harrison Chase. *LangChain*. Oct. 2022. URL: <https://github.com/langchain-ai/langchain>.
- [9] OpenAI. “Function calling”. <https://platform.openai.com/docs/guides/function-calling>. Accessed: 2025-06-17. 2024.
- [10] OpenAI. “Code Interpreter”. <https://openai.com/blog/chatgpt-plugins#code-interpreter>. Accessed: 2025-06-17. 2023.
- [11] Lei Wang et al. “Plan-and-Solve Prompting: Improving Zero-Shot Chain-of-Thought Reasoning by Large Language Models”. 2023. arXiv: 2305.04091 [cs.CL]. URL: <https://arxiv.org/abs/2305.04091>.
- [12] Binfeng Xu et al. “ReWOO: Decoupling Reasoning from Observations for Efficient Augmented Language Models”. 2023. arXiv: 2305.18323 [cs.CL]. URL: <https://arxiv.org/abs/2305.18323>.
- [13] Xukun Liu et al. “ToolNet: Connecting Large Language Models with Massive Tools via Tool Graph”. 2024. arXiv: 2403.00839 [cs.AI]. URL: <https://arxiv.org/abs/2403.00839>.

- [14] Yu Du, Fangyun Wei, and Hongyang Zhang. “AnyTool: Self-Reflective, Hierarchical Agents for Large-Scale API Calls”. 2024. arXiv: 2402.04253 [cs.CL]. URL: <https://arxiv.org/abs/2402.04253>.
- [15] Jiashuo Sun et al. “Think-on-Graph: Deep and Responsible Reasoning of Large Language Model on Knowledge Graph”. 2024. arXiv: 2307.07697 [cs.CL]. URL: <https://arxiv.org/abs/2307.07697>.
- [16] Yanming Liu et al. “Tool-Planner: Task Planning with Clusters across Multiple Tools”. 2025. arXiv: 2406.03807 [cs.AI]. URL: <https://arxiv.org/abs/2406.03807>.
- [17] Kevin Chen et al. “Reinforcement Learning for Long-Horizon Interactive LLM Agents”. 2025. arXiv: 2502.01600 [cs.LG]. URL: <https://arxiv.org/abs/2502.01600>.
- [18] John Yang et al. “InterCode: Standardizing and Benchmarking Interactive Coding with Execution Feedback”. In: *ArXiv abs/2306.14898* (2023). URL: <https://api.semanticscholar.org/CorpusID:259262186>.
- [19] Yifan Song et al. “RestGPT: Connecting Large Language Models with Real-World RESTful APIs”. 2023. arXiv: 2306.06624 [cs.CL]. URL: <https://arxiv.org/abs/2306.06624>.
- [20] Harsh Trivedi et al. “AppWorld: A Controllable World of Apps and People for Benchmarking Interactive Coding Agents”. 2024. arXiv: 2407.18901 [cs.SE]. URL: <https://arxiv.org/abs/2407.18901>.
- [21] *LangGraph*. Accessed: 2025-06-17. URL: <https://github.com/langchain-ai/langgraph>.
- [22] Noah Shinn et al. “Reflexion: language agents with verbal reinforcement learning”. In: *Neural Information Processing Systems*. 2023. URL: <https://api.semanticscholar.org/CorpusID:258833055>.
- [23] OpenAI. “Parallel Function Calling”. <https://platform.openai.com/docs/guides/function-calling#parallel-function-calling>. Accessed: 2025-06-17. 2024.
- [24] Sehoon Kim et al. “An LLM Compiler for Parallel Function Calling”. 2024. arXiv: 2312.04511 [cs.CL]. URL: <https://arxiv.org/abs/2312.04511>.
- [25] Bohan Chen and Andrea L. Bertozzi. “AutoKG: Efficient Automated Knowledge Graph Generation for Language Models”. 2023. arXiv: 2311.14740 [cs.CL]. URL: <https://arxiv.org/abs/2311.14740>.

附錄

A Prompts

A.1 計畫生成 Prompt

Extract steps from a given question into a concise and logical plan.
Identify if each step is an "(Execute API step)", "(Process step)", or "(Stop step)", and position this classification at the start of the step.
Focus solely on the API call sequence. Based on the available APIs, select the
→ appropriate API and generate a plan to execute the task.
Each step must utilize only one API or none, and if data from another step is
→ required, create a separate step to process that data.
Ensure steps are clear, actionable, and follow a logical sequence from start to
→ finish.
Use numbered steps and include "Using data (step 1),(step 2)...(step n)" for
→ steps requiring data from previous steps.
Steps cannot prompt for user input and must utilize only the provided APIs.
You cannot use variables; only the output values from previous steps can be
→ used.
Prefer collecting the items into a list first and then processing them in
→ batch, rather than using a loop.
Avoid calling APIs to fetch all data within a single step. Instead, include
→ appropriate constraints or filters in the step to limit the data retrieved.

Steps

1. ****Identify the Objective****: Clearly understand the task and determine the
→ essential steps needed, ensuring that batch queries are prioritized within
→ execution-type steps.
2. ****List Required APIs****: Use the provided list to select relevant APIs to
→ retrieve or process necessary data.
3. ****Plan Creation****: Analyze the task to create a sequence of numbered steps,
→ starting with query-type APIs, and ending with execution-type APIs. Each
→ step should process or execute one task, clearly defining the data it uses
→ from previous steps. Batch API calls within a single Execute API step when
→ feasible.
4. ****Define Steps****:
 - ****API Step****: An "(Execute API step)" should call one or more APIs as a
→ batch and use preceding step outputs if needed. Clearly link data
→ requirements by referencing earlier steps.
 - ****Process Step****: If processing is required without an API, mark it as
→ "(Process step)". Indicate when data from previous steps is utilized.
 - ****Stop Step****: The final step must be marked as "(Stop step)" and state
→ the task's return value or indicate "No return value required" for
→ tasks with no explicit outputs.
 - ****Jump Steps****: If a loop is needed, add a corresponding "(Process step)"
→ with loop conditions, specifying the target step to jump to with "(Jump
→ step)".

Output Format

The output should be a JSON list of steps, where each step is a string. Each
→ string begins with the type of step "(Execute API step)", "(Process step)",
→ or "(Stop step)".

Examples

****Input:****

Task: "What is the title of the least-played song in my Spotify song library?"
APIs: ["show_playlist_library", "show_playlist", "show_song", "login"]

****Output:****

```

```json
[
 "1. (Execute API step) Login to Spotify using login API",
 "2. (Execute API step) Get playlists from my Spotify playlist library using
 ↳ show_playlist_library api. Using data (step 1)",
 "3. (Process step) Extract playlist IDs from step 2. Using data (step 2)",
 "4. (Execute API step) Get all songs as a batch for all playlist IDs using
 ↳ show_playlist. Using data (step 3)",
 "5. (Execute API step) Get all songs play counts using show_song APIs. Using
 ↳ data (step 4)",
 "6. (Process step) Identify the least-played song from the data compiled in
 ↳ step 5. Using data (step 5)",
 "7. (Stop step) Return the title of the least-played song from step 6. Using
 ↳ data (step 6)"
]
```

# Notes

- Ensure execution-type APIs are positioned in later steps when possible.
- Batch calls to multiple APIs in a single step when possible to improve
  ↳ efficiency.
- For process steps involving loops, specify the loop conditions and
  ↳ destination step.
- The last step must be marked as "(Stop step)" and clearly state if a return
  ↳ value is required or not.
- Ensure jump steps are strictly limited to (Process step), and they will not
  ↳ appear elsewhere in the plan.
- Ensure that every step is categorized as exactly one of the following:
  ↳ "(Execute API step)", "(Process step)", or "(Stop step)". Jump steps must
  ↳ also be classified as (Process step).
- Prefer collecting the items into a list first and then processing them in
  ↳ batch, rather than using a loop.
- Stop step should only in the last step, and it should not be used in the
  ↳ middle of the plan.

```

A.2 驗證計畫內容 Prompt

Determine the missing information necessary to complete a given plan based on
 ↳ existing memory.
 If any values are missing, output the missing value names and reasons in JSON.
 If nothing is missing, output an empty JSON: ```json{}```.

Steps

1. ****Understand the Plan and Memory:****
 - Read through the provided plan thoroughly.
 - Review the memory information available.
2. ****Identify Missing Elements:****
 - Go step-by-step through the plan.
 - Determine if any step requires information that is absent from the memory.
3. ****Explain Missing Values:****
 - For each missing value, clearly articulate why it is needed for the plan's
 ↳ completion.
4. ****Output as JSON:****
 - Structure the necessary output in JSON format, including all missing values
 ↳ and their reasons if there are any.

Output Format

- JSON format.
- If values are missing, include the missing value names as keys and reasons as
 ↳ values.

```

- If no values are missing, output an empty JSON: ``json{}``.

# Examples

**Example 1:**

- **Plan:**
1. (Process data) Filter contacts whose age is over 20
...

- **Memory:**
``json
{"address": "Taiwan", "username": "Monkey"}

- **Output:**
- **Reasoning:** Since the memory does not contain contact information, it is
  ↳ not possible to complete step 1.
- **JSON Output:**
``json
{
  "contact list": "Contact information is required to achieve the goal, so it
  ↳ is necessary to retrieve the contact list"
}
``

**Example 2:**

- **Plan:**
1. List all product using list_products API
2. (Process data) Filter products that price is over 100 from step 1
3. (Process data) Return the total number of products from step 2

- **Memory:** : {}

- **Output:**
- **Reasoning:** The memory does not contain information about products, but it
  ↳ can be retrieved using the list_products API. Therefore, no missing
  ↳ information is identified.
- **JSON Output:**
``json
{}
``

# Notes

- Ensure the accuracy of reasoning before forming conclusions.
- Consider similar cases with different missing data to practice determining
  ↳ the correct output.
- Pay attention to distinguishing between different kinds of memory
  ↳ information, ensuring nothing important is overlooked.`

```

A.3 終止步驟分析結果

Identify if a task requires returning a final answer by analyzing if an answer

- ↳ is necessary based on past execution experience or specific task
- ↳ characteristics.

For tasks where an answer is required, such as queries that involve retrieving

- ↳ or computing specific information, provide the resultant answer in a
- ↳ specified format. For tasks that involve actions with no distinct resultant
- ↳ answer, such as commands or instructions, indicate that no answer is
- ↳ needed.

Steps

1. **Analyze the Task:** Evaluate whether the task requires a distinct
 - ↳ response. Queries typically require answers, whereas action-based commands
 - ↳ may not.

2. ****Review Execution Results:**** Examine the executed results to identify any
 ↪ present answers relevant to the task.
3. ****Determine Requirement:**** Decide if the task inherently requires an answer
 ↪ based on its type and previous experience.
4. ****Construct JSON Output:**** Format your determination as a JSON object.

Output Format

Provide the output in the JSON format:

```
```json
{
 "answer": "[answer here]"
}
```

- If the task requires a final answer, fill in the `[answer here]` with the  
     ↪ relevant detail.

- If no final answer is required, return:

```
```json
{
  "answer": "null"
}
```

Examples

****Example 1:****

```
- **Input:**
- Task: What is the least-played song in my playlist library?
- Executed Results: {"least-played_song": "hello world"}
- **Output:**
```json
{"answer": "hello world"}
```

##### **\*\*Example 2:\*\***

```
- **Input:**
- Task: Play the least-played song in my playlist library.
- Executed Results: {}
- **Output:**
```json
{"answer": "null"}
```

Notes

- A task focused on retrieving or calculating information usually requires an
 ↪ answer.
- A task that is directive or predominantly action-oriented generally does not
 ↪ return a visible answer.
- The output must be enclosed within ```json ```.

A.4 API 内部参数相依性分析 Prompt

Determine if there exists any dependency between given parameters.
 If a dependency is identified, the dependent parameters must be considered
 ↪ together and output in a JSON format as specified.
 Only a single dependency can exist for each parameter.
 Hidden relationships should also be considered—for example, sending money to
 ↪ someone implies a connection between the money and the person.

Steps

1. ****Identify Dependencies**:** Examine each parameter to determine if there is a
 ↪ dependency with another parameter.

2. ****Group Dependent Parameters****: If a dependency is found, group them
 - ↳ together in a nested JSON array.
3. ****Output in JSON Format****: Ensure that each set of dependent parameters is
 - ↳ placed within its own array, and all such arrays are collected together in
 - ↳ a JSON structure. Independent parameters should not be included in this
 - ↳ output.

Output Format

- JSON array of arrays, where each sub-array includes parameters that must be
 - ↳ considered together due to their dependency.

Examples

- ***Input***:
 - API: file_system.move_file
 - parameters: ["source location", "target location", "access_token"].
- ***Output***:

Since the source and destination locations for each file to be moved are

 - ↳ different, the source location and target location are considered to have a
 - ↳ dependency.

```

[[{"source location", "target location"}]]
      
```

Notes

- If no dependencies exist among parameters, the output should be an empty JSON
 - ↳ array: ```json[]```.
- Ensure accuracy in determining dependencies as each parameter will have at
 - ↳ most one dependency.

A.5 可調整參數生成 Prompt

Determine whether each modifiable parameter of an API needs to be changed based

- ↳ on the task goal, API documentation, and any other relevant contexts.

Provide a specific result for each modifiable parameter, or return null if a

- ↳ parameter does not require modification.

Follow the parameter description in the API documentation to determine

- ↳ appropriate values.

For query-type APIs, aim to modify the parameters to maximize results, such as

- ↳ toggling values or adjusting pagination.

Pagination parameters like ``page_index`` and ``page_limit`` are interdependent;

- ↳ changing one should lead to an appropriate adjustment of the other.

The output should only include a set of modifiable parameters.

Steps

1. ****Identify Task Goal****
 - Examine the task goal to understand desired outcomes from API calls.
2. ****Review API Documentation****
 - Thoroughly read the API name, description, and parameters to understand their
 - ↳ functions and relationships.
3. ****Analyze Modifiable Parameters****
 - For each modifiable parameter, decide whether it needs adjustment based on
 - ↳ the task goal.
 - Follow specific guidelines for query-type APIs or pagination parameters.
4. ****Parameter Modification****
 - Determine the appropriate value for each modifiable parameter:
 - Adjust parameters like ``page_limit`` to maximize results.
 - Ensure pagination parameters are coordinated.
5. ****Output Formation****
 - Compose a structured output detailing the modification of each parameter or
 - ↳ specifying null if no change is needed.

Output Format

Provide a JSON object with the values of each modifiable parameter. If a

- ↳ parameter does not require modification, assign it a null value.

```
```json
{
 "parameter_with_value": "value",
 "parameter_with_null": null,
 ...
}
```
```

Example

****Input:****

```
- **Goal:** Retrieve all liked albums
- **API Documentation:**
- **Name:** show_liked_albums
- **Description:** "Get a list of albums you have liked."
- **Parameters:** 'access_token', 'page_index', 'page_limit', 'sort_by'
- **Modifiable Parameters:** ["page_index", "page_limit", "sort_by"]
```

****Reasoning:****

To achieve the goal of retrieving all liked albums, modify parameters to fetch
 ↳ more results. Therefore, increase the `page\_limit` to the maximum and
 ↳ adjust the `page\_index` if needed to start from the beginning.

****Output:****

```
```json
{
 "page_index": 0,
 "page_limit": 20,
 "sort_by": "-liked_at"
}
```
```

Notes

- Ensure that pagination parameters are adjusted together to reflect the
 ↳ appropriate results span.
- Consider toggling boolean parameters where applicable to cover all possible
 ↳ result scenarios.
- Always adhere to the API's specified behaviors and constraints when adjusting
 ↳ parameters.

A.6 可調整參數生成 Prompt

Determine whether each modifiable parameter of an API should be changed based
 ↳ on the task goal, API documentation, memory, and past call experience.
 The output should only include a set of modifiable parameters.

Use the following approach:

- Refer to the parameters' descriptions in the API documentation to decide
 ↳ their values.
- If error messages from past calls are related to modifiable parameters,
 ↳ attempt to modify the relevant parameters.
- If the error messages are unrelated to modifiable parameters, return an empty
 ↳ JSON object.
- If the API is a command-execution type and the call experience indicates
 ↳ success, return null for all parameters.
- For query type APIs, adjust parameters to maximize results (e.g., toggle
 ↳ 'is_public' or maximize 'page_limit').
- Adjust pagination parameters ('page_index' and 'page_limit') logically in
 ↳ response to their interdependencies.

Steps

1. ****Identify API Type**:** Determine if the API is command-execution or query.

2. ****Error Analysis****: Examine past call results for errors related to
 - ↳ modifiable parameters.
3. ****Parameter Modification****: Adjust modifiable parameters to meet the task
 - ↳ goal using the API documentation:
 - For command-execution APIs with successful past calls, set all parameters to null.
 - For query APIs, maximize or adjust parameters to broaden result sets (e.g.,
 - ↳ maximize 'page_limit', adjust 'page_index').
4. ****Output Decision****: Based on the above analysis, decide if parameters
 - ↳ should be modified or if null/empty JSON should be returned.

Output Format

The output should be in JSON format. Each key represents a parameter name, and

- ↳ the value should be adjusted based on the reasoning, or null if no change
- ↳ is needed. If no modification is required and errors are unrelated, output
- ↳ an empty JSON object.

Examples

Example 1

****Input****

```
- **Goal**:
```

```
- **API Documentation**:
```

```
- **Name**:
```

```
- **Description**:
```

```
- **Parameters**:
```

```
- **Modifiable Parameters**:
```

```
- **Call Experience**:
```

```
```json
```

```
[{"Parameters used": {"access_token": "", "page_index": 0, "page_limit": 20,
```

```
 ↳ "sort_by": "-liked_at"}, "result": "Exception: not a valid access_token"}]
```

```
```
```

****Reasoning****:

The API was called with an invalid 'access_token', which cannot be modified.

- ↳ The issue does not relate to modifiable parameters.

****Output****:

```
```json
```

```
{}
```

```
```
```

Example 2

****Input****

```
- **Goal**:
```

```
- **API Documentation**:
```

```
- **Name**:
```

```
- **Description**:
```

```
- **Parameters**:
```

```
- **Modifiable Parameters**:
```

```
- **Call Experience**:
```

```
```json
```

```
[{"Parameters used": {"access_token": "ey8632231568", "page_index": 0,
```

```
 ↳ "page_limit": 20, "sort_by": "-liked_at"}, "result": ["album1", "album2",
```

```
 ↳ ..., "album20"]}]
```

```
```
```

****Reasoning****:

Only 20 albums retrieved; pagination needs adjustment to obtain more albums.

****Output****:

```
```json
```

```
{
```

```
 "page_index": 1,
```

```
 "page_limit": 20,
```

```
 "sort_by": "-liked_at"
```

```
}```
```

```
}...
```

### Example 3

**\*\*Input:\*\***

```
- **Goal**: Retrieve all liked albums
- **API Documentation**:
- **Name**: show_liked_albums
- **Description**: "Get a list of albums you have liked."
- **Parameters**: 'access_token', 'page_index', 'page_limit', 'sort_by'
- **Modifiable Parameters**: ["page_index", "page_limit", "sort_by"]
- **Call Experience**:
```json
[{"Parameters used": {"access_token": "ey8632231568", "page_index": 0,
  ↪ "page_limit": 20, "sort_by": "-liked_at"}, "result": ["album1", "album2",
  ↪ "album3"]}]
```

****Reasoning**:**

The past call showed fewer albums than the page limit, indicating all results
↪ are retrieved.

****Output**:**

```
```json
{
 "page_index": null,
 "page_limit": null,
 "sort_by": null
}
```

## A.7 驗證 API 執行結果 Prompt

Analyze the parameters used and the results of the executed API to determine  
↪ whether the API was successfully executed according to the given task  
↪ instructions and step goal.

If the API call completes successfully but the step goal is still missing  
↪ required information, append '(missing api)' to the end of the reason.

If the operation fails because the item already exists when adding, or because  
↪ it no longer exists when deleting, please return success as true.

If the error message is caused by an incorrect required parameter, please  
↪ clearly indicate the erroneous parameter and the missing content and append  
↪ '(missing api)' to the end of the reason.

Consider the success of the operation based on the following evaluation:

- If the task was completed successfully or minor acceptable errors are  
↪ present, return {"success": true}.
- If there are errors that prevent the task from being completed, return  
↪ {"success": false, "reason": ["reason\_here"]}.

Maintain JSON formatting in the output at all times.

# Steps

1. **\*\*Interpret Task and Step Goal\*\***: Understand what the task aims to achieve  
↪ and the specific goal of the step in question.
2. **\*\*Analyze API Parameters\*\***: Review the parameters used for the API call to  
↪ ensure correctness and relevance to the task.
3. **\*\*Evaluate Results\*\***: Assess the `result` field of the API call to  
↪ understand whether each action was successful.
4. **\*\*Determine Success\*\***: Compare the parameter utilizations and results  
↪ against the task and step goals:
  - If the majority of actions in the result are successful and the failures are  
↪ acceptable (e.g., already-deleted entities), consider it a success.

- If significant errors (e.g., wrong credentials, expired payment methods) are present, categorize it as unsuccessful.
5. **\*\*Output Conclusion\*\***: Use a clear JSON structure to report success or failure, with reasons explained if applicable.

#### # Output Format

- The output should always be in a JSON structure: ``{"success": true}`` for success, or ``{"success": false, "reason": ["reason_here"]}`` for failure.
- Include detailed reasons for failure where applicable.

#### # Examples

##### ## Example 1

**\*\*Input\*\***:

Task: Delete all emails dated before today

Step Goal: Delete all emails listed in step 3

API in Execution: gmail.delete\_email

API Call Experience:

```
```json
[
  {
    "Parameters used": {"email_id": [1,2,3,4,5,6,7,8,9], "access_token":
      ↪ "98456132"},
    "result": ["success deleted!", "success deleted!", "success deleted!", "mail id
      ↪ not found", "success deleted!", "success deleted!", "success deleted!",
      ↪ "success deleted!", "success deleted!"]
  }
],
```
```

**\*\*Reason\*\***: The command for email ID 4 failed, but the error indicates that the  
 ↪ mail ID was not found, which could mean the email was already deleted.  
 ↪ Therefore, we consider the operation successful.

**\*\*Output\*\***:

```
```json
{"success": true}
```
```

##### ## Example 2

**\*\*Input\*\***:

Task: Purchase all items from my favorites list

Step Goal: Use the list from step 5 and the payment info from step 3 to pay for  
 ↪ and buy all items in the list

API in Execution: amazon.buy\_item

API Call Experience:

```
```json
[
  {
    "Parameters used": {"item_id": [239, 124, 982], "access_token": "98456132",
      ↪ "payment_card_id": 127},
    "result": ["payment card has been expired!", "payment card has been expired!",
      ↪ "payment card has been expired!"]
  }
],
```
```

**\*\*Reason\*\***: The output indicates the credit card used (payment\_card\_id 127) has  
 ↪ expired. Therefore, the API did not execute successfully.

**\*\*Output\*\***:

```
```json
{
  "success": false,
  "reason": "The API failed because the credit card with payment_card_id 127 has
    ↪ expired."
}
```

```
}..
```

```
# Notes
```

- When encountering multiple results, consider the overall outcome rather than
 - focusing solely on individual failures.
- Errors indicating non-critical issues (e.g., items already absent or
 - previously deleted) may still result in success if the overarching task is
 - achieved.

A.8 驗證已有資訊是否能夠進行資料處理 Prompt

Evaluate the data in Memory and Previous Steps Result to determine if there is

→ sufficient information to complete the objective of the Current step.

If the data is sufficient, indicate this in JSON format. If there is

→ insufficient information, provide a reason in JSON format.

If the information cannot be found, try searching through the API collection

→ for any available data to return.

In the "reason" field, specify which data could not be found and suggest which

→ type of API could be used to retrieve it.

API Collection: {apis}

If the insufficient information can't be found in the API collection, append

→ the string "(missing api)" to the end of the "reason" field.

This is apps data {tools.get_apps()}, you can use it to think about missing api

→ is from which app.

```
# Steps
```

1. Review the data available in Memory and Previous Steps Result relevant to

→ the Current step.

2. Assess whether the data is comprehensive enough to achieve the objective of

→ the Current step.

3. If sufficient, confirm completion.

4. If insufficient, identify and explain the missing information.

```
# Output Format
```

Provide your conclusion in JSON format:

- If sufficient information is available, return:

```
```json
{"enough": true}
```

- If information is insufficient, return:

```
```json
{"enough": false, "reason": "[explanation here]"}
```

```
# Notes
```

- Ensure the reason provided for insufficient information is clear and concise.
- Carefully check all available data before concluding to minimize incorrect
 - assessments of sufficiency.

A.9 Python 程式碼生成 Prompt

Generate a complete Python script to process input data and print the results.

Ensure the Python code is clean and readable. No comments should be included in

→ the code.

The data from Steps Result and Memory will be in the "steps_result" and

→ "memory" variables, respectively.

Python code should use "steps_result" and "memory" variables directly to access

→ the data. Don't assign them again.

The final output should include a label, such as "total_funds," and its
→ corresponding value.
Output should be in Python dictionary format.

Only basic Python libraries such as `json`, `pandas`, or `numpy` may be used.
→ Do not use any external libraries or APIs like `ast` or `requests`. If this
→ task concludes with a (Jump step), ensure the code outputs "(jump to step
→ n)" at the end, where n is the step number. Parentheses are required.

Output Format

Ensure that the Python script includes code to:

- Import necessary libraries.
- Process the input data using "steps_result" and "memory" directly.
- Calculate or derive any requested results.
- Print the results in a readable format.

A.10 驗證 python 執行結果 Prompt

Determine if the data in the step result meets the objective of the specified
→ step. If it does, output a JSON indicating success. If not, output a JSON
→ indicating failure along with a detailed reason in the "reason" field.
Don't judge about data type. Only care about data contents.
If the output label has the same meaning as the required label, the result is
→ acceptable
If the step goal is jump step, please make sure the output is "(jump to step
→ n)", where n is the step number to jump to. Parentheses are required.

Steps

1. ****Understand the Task and Step Goal****:
- Interpret the task and its specific objective for the step.
2. ****Analyze the Step Result****:
- Evaluate the provided data in the step result to determine if it meets all
→ criteria defined in the step goal.
3. ****Determine the Outcome****:
- If all criteria are met, conclude with a success output.
- If criteria are not met, identify missing elements or discrepancies.
4. ****Output the Result****:
- Create a JSON output that either confirms success or explains failure.

Output Format

```
- If successful:
  ``json
  {
    "result": true
  }
  ...

- If unsuccessful:
  ``json
  {
    "result": false,
    "reason": "[Explain the discrepancy or missing data here]"
  }
  ...
```

Examples

****Example 1:****

Input:

- ****Task****: Obtain the user's roommate's address, phone number, and birthday.

- ****Step Goal****: Filter the roommate's address, phone number, and birthday from
 - ↳ the dataset of friends extracted in step 2. The list of roommates can be
 - ↳ obtained from step 3.
- ****Step Result****: [{"name": "Peggy", "phone": "00878", "birthday": "1999/5/11"}, {"name": "Ada", "phone": "00629", "birthday": "1998/12/23"}]

Output:

- ****Reasoning****: Evaluate if the result includes all required information
 - ↳ (address, phone, birthday).
- ****Conclusion****: The data missing the address.

```
```json
{
 "result": false,
 "reason": "Missing address information"
}
```
```

Notes

- Ensure thorough examination of the data against the step goal.
- Clearly explain any deficiencies in the "reason" field in a concise manner.
- Maintain JSON formatting throughout all outputs.

A.11 重新規劃 Prompt

Extract steps from a given question into a concise and logical plan,

- ↳ identifying each step as an "(Execute API step)", "(Process step)", or
- ↳ "(Stop step)" at the beginning.

Focus on performing batch calls within a single Execute API step whenever

- ↳ possible, using only the output values from previous steps.

The sequence should clearly guide API calls, ignoring direct use of variables,

- ↳ while ensuring steps contain only one API or none and maintaining a clear,
- ↳ sequential flow without sub-steps.

Resolution must include adjustments based on previous failures if needed.

Jump steps are strictly limited to the (Process step) — they will not appear

- ↳ in (Execute API step) or (Stop step).

Use numbered steps and include "Using data (step 1),(step 2)...(step n)" for

- ↳ steps requiring data from previous steps.

Prefer collecting the items into a list first and then processing them in

- ↳ batch, rather than using a loop.

Avoid calling APIs to fetch all data within a single step. Instead, include

- ↳ appropriate constraints or filters in the step to limit the data retrieved.

You have the ability to operate on personal state information related to the

- ↳ service, such as adding a payment card or an address. This type of personal
- ↳ state information will be provided in Memory. You can add an (Execute API
- ↳ step) to request the addition of such information.

Steps

1. ****Identify the Objective****: Clearly understand the task and determine the
 - ↳ essential steps needed, ensuring that batch queries are prioritized within
 - ↳ execution-type steps.
2. ****List Required APIs****: Use the provided list to select relevant APIs to
 - ↳ retrieve or process necessary data.
3. ****Define Steps****:
 - Assign a label at the start of each step, specifying it as either an
 - ↳ "(Execute API step)", "(Process step)", or "(Stop step)".
 - Sequence API calls to optimize batch execution wherever possible, utilizing
 - ↳ only output from prior steps.
 - If information is unavailable and assumed necessary, include an additional
 - ↳ step "(Execute API step) missing_api" to retrieve data.
 - Specify loop conditions and jump points exclusively in (Process step) for any
 - ↳ repeated execution.
 - Prefer collecting the items into a list first and then processing them in
 - ↳ batch, rather than using a loop.

4. ****Execution of APIs****: Ensure batch execution-type API calls finalize any
 - ↳ data manipulations or service state changes.
5. ****Error Handling****: In case of failed execution:
 - Re-evaluate and restructure starting from the failed step.
 - Mark successfully executed steps with '(Pass)'. However, if the error reason
 - ↳ indicates that a prior step has failed, the step should not be marked as
 - ↳ '(Pass)'
 - Refresh data calls or correct processing logic as needed for any data
 - ↳ acquisition or processing errors.
6. ****Final Step****: The last step must indicate "(Stop step)" with a statement
 - ↳ like "Return [result]" or "No return value required".

Output Format

Output should be a JSON array including steps with labels, logical sequences,
 ↳ and necessary notes on data usage, as per the given examples.

Examples

****Input:****

Task: "What is the title of the least-played song in my Spotify song library?"

Previous Plan Steps:

- [
 - "1. (Execute api step) Get playlists from my Spotify playlist library using
 ↳ show_playlist_library api",
 - "2. (Process step) Extract playlist IDs from step 1. Using data (step 1)",
 - "3. (Execute api step) Get all songs from all playlists using show_playlist
 ↳ for each playlist ID from step 2. Using data (step 2)",
 - "4. (Process step) Identify the least-played song from the compiled list in
 ↳ step 3. Using data (step 3)",
 - "5. (Stop step) Return the title of the least-played song from step 5.
 ↳ Using data (step 5)"

Error Step: "4"

Error Reason: "step 3 does not contain the play count of the songs, so it is
 ↳ not possible to identify the least-played song"

APIs: ["show_playlist_library", "show_playlist", "show_song"]

****Output:****

Reason: "The previous plan did not include a step to retrieve the play counts
 ↳ of the songs. show_song API should be used to get the play counts of the
 ↳ songs in step 4.",

...json

- [
 - "1. (Execute api step) Get playlists from my Spotify playlist library using
 ↳ show_playlist_library api (Pass)",
 - "2. (Process step) Extract playlist IDs from step 1. Using data (step 1)
 ↳ (Pass)",
 - "3. (Execute api step) Get all songs from all playlists using show_playlist
 ↳ for each playlist ID from step 2 Using data (step 2) (Pass)",
 - "4. (Execute api step) Compile a list of all songs and their play counts
 ↳ using show_song for each song ID using step 3. Using data (step 3)",
 - "5. (Process step) Identify the least-played song from the compiled list in
 ↳ step 4. Using data (step 4)",
 - "6. (Stop step) Return the title of the least-played song from step 5.
 ↳ Using data (step 5)"

...
 ...

Notes

- Ensure each step leverages outputs from previous steps and maintains a clear
 ↳ logical progression.
- Address any missing data through an additional execute step to retrieve it.
- When addressing failed steps, reassess the plan from the point of failure,
 ↳ with potential refactoring from earlier points if needed.

- Add "(Missing data)" to highlight steps addressing absent data.
- Ensure jump steps are strictly limited to (Process step), and they will not
 ↪ appear elsewhere in the plan.

B 程式碼

B.1 自動構建知識圖譜程式碼

API 節點生成程式碼

```
class ApiNode:
    def __init__(self, app: str, labels :set, name :str, method :str, url :
        ↪ str, document :str, description :str):
        self.app = app
        self.labels = labels
        self.name = name
        self.method = method
        self.url = url
        self.document = document
        self.description = description

def get_apis():
    apis: list[ApiNode] = []
    # 從文件讀取 API 文檔
    api_doc_path = os.path.join(os.getcwd(), "data/api_docs/standard")
    for root, dirs, files in os.walk(api_doc_path):
        for file in files:
            file_path = os.path.join(root, file)
            with open(file_path, "r") as f:
                json_object = json.load(f)
                for key, value in json_object.items():
                    labels = ["API", value["app_name"]]
                    name = key
                    method = value["method"]
                    path = value["path"]
                    document = json.dumps(value)
                    description = value["description"]
                    if method == "GET":
                        labels += ["query"]
                    api_node = ApiNode(value["app_name"], set(labels), name,
                        ↪ method, path, document, description)
                    apis.append(api_node)

    return apis

def create_api_node():
    apis = get_apis()
    graph = Graph(NEO4J_URI, auth=(USERNAME, PASSWORD))
    tx = graph.begin()
    for api in apis:
        node = Node(*api.labels, name=api.name, method=api.method, url=api.url,
            ↪ document=str(api.document), description=api.description)
        tx.create(node)
        node_app = Node("App", name=api.app)
        tx.merge(node_app, "App", "name")
    graph.commit(tx)
```

API 參數與關鍵字節點和關係建立程式碼

```
def create_relation_api_keyword():
    api_sys_msg = \
        """
        Identify keywords within the name and description of an API, and
        ↪ describe the relationship between each keyword and the API in one
        ↪ sentence.

        # Steps

        1. **Analyze the API Name**: Extract key terms from the API name that
        ↪ are crucial to its function.
        2. **Review the Description**: Identify keywords within the description
        ↪ that highlight the core action or function of the API.
        3. **Keyword Relationship**: For each keyword identified, write a
        ↪ single sentence that clearly describes how the keyword is related
        ↪ to the API.
        4. **Structure the Output**: Format the final result as a JSON object
        ↪ with each keyword.

        # Output Format

        - Output should be in JSON format as a list of objects.
        - Each object should have a single key-value pair where the keyword is
        ↪ the "keyword" and the relationship is the "relation."
        - All keywords must be in their original lowercase English form.

        # Examples

        ## Example 1
        **API**: `add_album_to_library`
        **Description**: `Add an album to the user's album library.`

        **Output**:
        ```json
 [
 {"keyword": "album", "relation": "The API allows adding an album to
 ↪ the user's collection."},
 {"keyword": "library", "relation": "Stores the album in the user's
 ↪ library."},
 {"keyword": "user", "relation": "Extends the user's personal album
 ↪ library."}
]
        ```

        # Notes

        - Ensure each keyword relationship is accurate and directly related to
        ↪ the function of the API.
        - Avoid generic terms that do not contribute meaningful context to the
        ↪ API's purpose.
        - Focus on operational keywords that highlight specific actions or
        ↪ services provided by the API.
        """

    graph = Graph(NEO4J_URI, auth=(USERNAME, PASSWORD))
    matcher = NodeMatcher(graph)
    nodes = matcher.match("API")
    for node in nodes:
        # 從所有 API 節點中取出 name 和 description，並呼叫 LLM 輸出關鍵字
        print("API Name: ", node["name"])
        print("API Description: ", node["description"])
        time.sleep(0.5)
        human_msg = f"""API:{ node["name"]} Description:
        ↪ {node["description"]}"""
        json_result = tools.get_json_from_llm(api_sys_msg, human_msg,
        ↪ float(NORMAL_LLM_TEMP), LLM_MODEL)
        print(json_result)
        for item in json_result:
```

```

        kw = str(item["keyword"]).lower()
        value = str(item["relation"])
        tx = graph.begin()
        kw_node = Node("Keyword", name=kw)
        tx.merge(kw_node, "Keyword", "name")
        relation = Relationship(kw_node, "keyword_relation", node,
                               ↪ relation=value)
        tx.create(relation)
        graph.commit(tx)

def create_relation_param_response():
    graph = Graph(NEO4J_URI, auth=(USERNAME, PASSWORD))
    matcher = NodeMatcher(graph)
    nodes = matcher.match("API", "supervisor")
    for node in nodes:
        # 從 API 節點中取出所有參數和參數的 description，並呼叫 LLM 輸出關鍵字
        api_doc = json.loads(node["document"])
        params = api_doc["parameters"]
        response_schema = api_doc["response_schemas"]["success"]
        params_name = []
        for param in params:
            params_name.append(param["name"])
            tx = graph.begin()
            print("Parameter name: ", param["name"])
            param_node = Node("Parameter", "supervisor", name=param["name"])
            relation = Relationship(node, "Require parameter", param_node,
                                   ↪ require=param["required"])
            tx.merge(param_node, "supervisor", "name")
            tx.create(relation)
            graph.commit(tx)

        if isinstance(response_schema, list):
            response_schema = response_schema[0]

        if response_schema == "string":
            continue

        for res, type in response_schema.items():
            if res in params_name:
                continue
            tx = graph.begin()
            ns = matcher.match("Parameter", name=res)
            if ns.count() == 0:
                res_node = Node("Parameter", "supervisor", name=res)
                relation = Relationship(node, "Response parameter", res_node)
                tx.merge(res_node, "supervisor", "name")
                tx.merge(relation, "Response parameter")
            else:
                for n in ns:
                    relation = Relationship(node, "Response parameter", n)
                    tx.merge(relation, "Response parameter")
            graph.commit(tx)
    print("done")

```